Retail Sales Analysis with Python and SQL - Using a real-world dataset

Author: Georgios Gerakis

LinkedIn: https://www.linkedin.com/in/georgios-gerakis/

Project Goal: Help an online retailer understand its past performance: top-selling products, customer activity and revenue trends.

I will use Python to clean the data, perform EDA, RFM customer segmentation with K-means, Market Basket Analysis with FP-Growth, then use SQL queries for further analysis.

Python

Data Cleaning

Load and preview the data.

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
import seaborn as sns
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from sklearn.decomposition import PCA
import sqlite3
from mlxtend.frequent_patterns import association_rules, fpgrowth
import warnings
import io, requests

warnings.filterwarnings("ignore", category=DeprecationWarning)
```

In [ ]:
```
url= "https://raw.githubusercontent.com/gerakisg/UK_Online_Retail_Transactions/main/data/Onli

try:
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    original_df = pd.read_excel(io.BytesIO(r.content))
except Exception as e:
    raise FileNotFoundError(
        f"Failed to download from GitHub URL.\nOriginal error: {e}"
    )
```

In [ ]:
```
df = original_df

df.head()
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

---

InvoiceNo: Invoice number. Nominal, a 6-digit integral number uniquely assigned to each transaction. If this code starts with letter 'c', it indicates a cancellation.

StockCode: Product (item) code. Nominal, a 5-digit integral number uniquely assigned to each distinct product.

Description: Product (item) name. Nominal.

Quantity: The quantities of each product (item) per transaction. Numeric.

InvoiceDate: Invoice Date and time. Numeric, the day and time when each transaction was generated.

UnitPrice: Unit price. Numeric, Product price per unit in sterling (British pounds).

CustomerID: Customer number. Nominal, a 5-digit integral number uniquely assigned to each customer.

Country: Country name. Nominal, the name of the country where each customer resides.

In [ ]:
```
print("Original Shape:", df.shape)
```

```
Original Shape: (541909, 8)
```

Our dataset has a total of 541909 rows (records) and 8 columns (features).

Remove duplicates

Let's delete duplicate records if they exist.

In [ ]:
```
df = df.drop_duplicates()
```

In [ ]:
```
print("New Shape:", df.shape)
```

```
New Shape: (536641, 8)
```

Rows decreased so duplicate records existed and were removed.

Feature types and null values.

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 536641 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype         
---  ------       --------------   -----         
 0   InvoiceNo    536641 non-null  object        
 1   StockCode    536641 non-null  object        
 2   Description  535187 non-null  object        
 3   Quantity     536641 non-null  int64         
 4   InvoiceDate  536641 non-null  datetime64[ns]
 5   UnitPrice    536641 non-null  float64       
 6   CustomerID   401604 non-null  float64       
 7   Country      536641 non-null  object        
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 36.8+ MB
```

We can observe that some columns are of dtype "object". These are categorical or text fields.

The columns of Dtype "int64" or "float64" are numerical columns with integer or decimal values, respectively.

The "InvoiceDate" column is of Dtype "datetime64[ns]", a timestamp value.

Importantly, we can see that the features "Description" and "CustomerID" have null values.

In [ ]:
```
df.isnull().sum()
```

Out[ ]:

	0
InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	135037
Country	0

dtype: int64

Here we can see exactly how many null values exist in each column.

Null Descriptions are not a problem for any particular analysis.

Null CustomerIDs are only a problem for customer-specific analysis but we can exclude them later instead of deleting them.

Numerical feature distributions

Let's add a boolean column for cancelled transactions for better readability.

In [ ]:
```
df = df.copy(deep=False)
df['IsCancelled'] = df['InvoiceNo'].astype(str).str.startswith('C')
```

In [ ]:
```
df.describe()
```

Out[ ]:

	Quantity	InvoiceDate	UnitPrice	CustomerID
count	536641.000000	536641	536641.000000	401604.000000
mean	9.620029	2011-07-04 08:57:06.087421952	4.632656	15281.160818
min	-80995.000000	2010-12-01 08:26:00	-11062.060000	12346.000000
25%	1.000000	2011-03-28 10:52:00	1.250000	13939.000000
50%	3.000000	2011-07-19 14:04:00	2.080000	15145.000000
75%	10.000000	2011-10-18 17:05:00	4.130000	16784.000000
max	80995.000000	2011-12-09 12:50:00	38970.000000	18287.000000
std	219.130156	NaN	97.233118	1714.006089

Some baseline statistics for our features such as minimum and maximum values, mean and IQR.

Some important observations are that 75% of Quantity values are lower than equal to 10, but the maximum is 80995.

The maximum UnitPrice value is 38970 but 75% of the values are below 5.

These outliers should be inspected in order to figure out if they are errors or valid records.

Invalid values

We also observe that 'Quantity' and 'UnitPrice' include zero and/or negative values.

Quantity negative values can be connected to cancelled transactions but that needs to be investigated. Zero values do not hold any logical meaning and thus are probably misleading and should be removed.

UnitPrice negative values do not hold any logical meaning and thus can be misleading and should be removed. Zero values can be attributed to promotions, bundles etc. and thus hold some logical value and could be part of an analysis.

Inspect records with Quantity == 0

In [ ]:
```
df[(df['Quantity'] == 0)]
```

Out[ ]:

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	IsCancel

Remove them

In [ ]:
```
df = df[(df['Quantity'] != 0)]
```

Let's Investigate records with negative Quantity values

In [ ]:
```
df[(df['Quantity'] < 0)]
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	14527.0	United Kingdom
154	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	2010-12-01 09:49:00	4.65	15311.0	United Kingdom
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	2010-12-01 10:24:00	1.65	17548.0	United Kingdom
236	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	2010-12-01 10:24:00	0.29	17548.0	United Kingdom
...	...	...	...	...	...	...	...	...
540449	C581490	23144	ZINC T-LIGHT HOLDER STARS SMALL	-11	2011-12-09 09:57:00	0.83	14397.0	United Kingdom
541541	C581499	M	Manual	-1	2011-12-09 10:28:00	224.69	15498.0	United Kingdom
541715	C581568	21258	VICTORIAN SEWING BOX LARGE	-5	2011-12-09 11:57:00	10.95	15311.0	United Kingdom
541716	C581569	84978	HANGING HEART JAR T-LIGHT HOLDER	-1	2011-12-09 11:58:00	1.25	17315.0	United Kingdom
541717	C581569	20979	36 PENCILS TUBE RED RETROSPOT	-5	2011-12-09 11:58:00	1.25	17315.0	United Kingdom

10587 rows × 9 columns

We can see that several of them are linked to cancelled transactions which should be included, but is it true for all of them?

In [ ]:
```
df[(df['Quantity'] < 0) & (~df['IsCancelled'])]
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
2406	536589	21777	NaN	-10	2010-12-01 16:50:00	0.0	NaN	United Kingdom
4347	536764	84952C	NaN	-38	2010-12-02 14:42:00	0.0	NaN	United Kingdom
7188	536996	22712	NaN	-20	2010-12-03 15:30:00	0.0	NaN	United Kingdom
7189	536997	22028	NaN	-20	2010-12-03 15:30:00	0.0	NaN	United Kingdom
7190	536998	85067	NaN	-6	2010-12-03 15:30:00	0.0	NaN	United Kingdom
...	...	...	...	...	...	...	...	...
535333	581210	23395	check	-26	2011-12-07 18:36:00	0.0	NaN	United Kingdom
535335	581212	22578	lost	-1050	2011-12-07 18:38:00	0.0	NaN	United Kingdom
535336	581213	22576	check	-30	2011-12-07 18:38:00	0.0	NaN	United Kingdom
536908	581226	23090	missing	-338	2011-12-08 09:56:00	0.0	NaN	United Kingdom
538919	581422	23169	smashed	-235	2011-12-08 15:24:00	0.0	NaN	United Kingdom

1336 rows × 9 columns

We find out that negative Quantity values are not always linked to cancelled transactions. Looking at the Description for these could provide some insight.

In [ ]:

```
df[
    (df['Quantity'] < 0) &
    (~df['IsCancelled']) &
    (df['Description'].notna())
  ]
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
7313	537032	21275	?	-30	2010-12-03 16:50:00	0.0	NaN	United Kingdom
13217	537425	84968F	check	-20	2010-12-06 15:35:00	0.0	NaN	United Kingdom
13218	537426	84968E	check	-35	2010-12-06 15:36:00	0.0	NaN	United Kingdom
13264	537432	35833G	damages	-43	2010-12-06 16:10:00	0.0	NaN	United Kingdom
21338	538072	22423	faulty	-13	2010-12-09 14:10:00	0.0	NaN	United Kingdom
...	...	...	...	...	...	...	...	...
535333	581210	23395	check	-26	2011-12-07 18:36:00	0.0	NaN	United Kingdom
535335	581212	22578	lost	-1050	2011-12-07 18:38:00	0.0	NaN	United Kingdom
535336	581213	22576	check	-30	2011-12-07 18:38:00	0.0	NaN	United Kingdom
536908	581226	23090	missing	-338	2011-12-08 09:56:00	0.0	NaN	United Kingdom
538919	581422	23169	smashed	-235	2011-12-08 15:24:00	0.0	NaN	United Kingdom

474 rows × 9 columns

Around two thirds of these transactions do not have a Description associated with them. For those that do have one, lets see what the most common ones are.

In [ ]:

```
df[
    (df['Quantity'] < 0) &
    (~df['IsCancelled']) &
    (df['Description'].notna())
]['Description'].value_counts().reset_index().head(10)
```

Out[ ]:

	Description	count
0	check	120
1	damages	45
2	damaged	42
3	?	41
4	sold as set on dotcom	20
5	Damaged	14
6	thrown away	9
7	Unsaleable, destroyed.	9
8	??	7
9	damages?	5

From these Descriptions we can gather that these records are of items that were damaged, destroyed etc.

This information could be utilized for a specific analysis.

It looks like several of these records have a UnitPrice equal to zero which means they would not be a problem for a revenue analysis. Also they have null CustomerIDs. Let's check if these conditions are both true for all of them.

In [ ]:
```
df[
    (df['Quantity'] < 0) &
    (~df['IsCancelled']) &
    ((df['UnitPrice'] != 0) | (df['CustomerID'].notna()))
  ]
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	IsCancel

We have confirmed that all of these cases have a UnitPrice of zero and a null CustomerID so we can carefully include them in the database and keep in mind to exclude null CustomerID records specifically when conducting customer analysis.

Let's see if there are any more records with a UnitPrice of zero

In [ ]:
```
df[(df['UnitPrice'] == 0) & (df['Quantity'] >= 0)]
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
622	536414	22139	NaN	56	2010-12-01 11:52:00	0.0	NaN	United Kingdom
1970	536545	21134	NaN	1	2010-12-01 14:32:00	0.0	NaN	United Kingdom
1971	536546	22145	NaN	1	2010-12-01 14:33:00	0.0	NaN	United Kingdom
1972	536547	37509	NaN	1	2010-12-01 14:33:00	0.0	NaN	United Kingdom
1987	536549	85226A	NaN	1	2010-12-01 14:34:00	0.0	NaN	United Kingdom
...	...	...	...	...	...	...	...	...
535334	581211	22142	check	14	2011-12-07 18:36:00	0.0	NaN	United Kingdom
536981	581234	72817	NaN	27	2011-12-08 10:33:00	0.0	NaN	United Kingdom
538504	581406	46000M	POLYESTER FILLER PAD 45x45cm	240	2011-12-08 13:58:00	0.0	NaN	United Kingdom
538505	581406	46000S	POLYESTER FILLER PAD 40x40cm	300	2011-12-08 13:58:00	0.0	NaN	United Kingdom
538554	581408	85175	NaN	20	2011-12-08 14:06:00	0.0	NaN	United Kingdom

1174 rows × 9 columns

Let's check the most common descriptions for these records.

In [ ]:
```
invalid_unitprice = df[(df['UnitPrice'] == 0) & (df['Quantity'] >= 0)].copy()
invalid_unitprice['Description'].value_counts().reset_index().head(10)
```

Out[ ]:

	Description	count
0	check	39
1	found	25
2	adjustment	14
3	FRENCH BLUE METAL DOOR SIGN 1	9
4	FRENCH BLUE METAL DOOR SIGN 8	8
5	amazon	8
6	Found	8
7	FRENCH BLUE METAL DOOR SIGN 3	7
8	RECIPE BOX PANTRY YELLOW DESIGN	7
9	OWL DOORSTOP	7

These Descriptions do not provide any valueable insights so we can just remove these records.

In [ ]:
```
df = df[(df['UnitPrice'] != 0) | (df['Quantity'] < 0)]
```

Inspect records with UnitPrice < 0.

In [ ]:
```
df[(df['UnitPrice'] < 0)]
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
299983	A563186	B	Adjust bad debt	1	2011-08-12 14:51:00	-11062.06	NaN	United Kingdom
299984	A563187	B	Adjust bad debt	1	2011-08-12 14:52:00	-11062.06	NaN	United Kingdom

These two records are not related to any sale and should be deleted.

We have seen that some StockCodes are not in the standard format. We should check what they are related to.

In [ ]:
```
df[~df['StockCode'].astype(str).str.match(r'^\d')]
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
45	536370	POST	POSTAGE	3	2010-12-01 08:45:00	18.00	12583.0	France
141	C536379	D	Discount	-1	2010-12-01 09:41:00	27.50	14527.0	United Kingdom
386	536403	POST	POSTAGE	1	2010-12-01 11:27:00	15.00	12791.0	Netherlands
1123	536527	POST	POSTAGE	1	2010-12-01 13:04:00	18.00	12662.0	Germany
1423	536540	C2	CARRIAGE	1	2010-12-01 14:05:00	50.00	14911.0	EIRE
...	...	...	...	...	...	...	...	...
541540	581498	DOT	DOTCOM POSTAGE	1	2011-12-09 10:26:00	1714.17	NaN	United Kingdom
541541	C581499	M	Manual	-1	2011-12-09 10:28:00	224.69	15498.0	United Kingdom
541730	581570	POST	POSTAGE	1	2011-12-09 11:59:00	18.00	12662.0	Germany
541767	581574	POST	POSTAGE	2	2011-12-09 12:09:00	18.00	12526.0	Germany
541768	581578	POST	POSTAGE	3	2011-12-09 12:16:00	18.00	12713.0	Germany

2971 rows × 9 columns

Let's get the most common Description for each of those StockCodes and sort by frequency.

In [ ]:
```
invalid_stockcodes = df[~df['StockCode'].astype(str).str.match(r'^\d')].copy()
```

```
invalid_stockcodes[['StockCode', 'Description']].value_counts().reset_index()
```

Out[ ]:

	StockCode	Description	count
0	POST	POSTAGE	1252
1	DOT	DOTCOM POSTAGE	707
2	M	Manual	560
3	C2	CARRIAGE	143
4	D	Discount	77
5	S	SAMPLES	62
6	BANK CHARGES	Bank Charges	37
7	AMAZONFEE	AMAZON FEE	34
8	CRUK	CRUK Commission	16
9	DCGSSGIRL	GIRLS PARTY BAG	13
10	DCGSSBOY	BOYS PARTY BAG	11
11	gift_0001_20	Dotcomgiftshop Gift Voucher £20.00	9
12	gift_0001_10	Dotcomgiftshop Gift Voucher £10.00	8
13	gift_0001_30	Dotcomgiftshop Gift Voucher £30.00	7
14	gift_0001_50	Dotcomgiftshop Gift Voucher £50.00	4
15	DCGS0003	BOXED GLASS ASHTRAY	4
16	gift_0001_40	Dotcomgiftshop Gift Voucher £40.00	3
17	B	Adjust bad debt	3
18	PADS	PADS TO MATCH ALL CUSHIONS	3
19	DCGS0076	SUNJAR LED NIGHT NIGHT LIGHT	2
20	DCGS0003	ebay	1
21	DCGS0069	ebay	1
22	DCGS0069	OOH LA LA DOGS COLLAR	1
23	DCGS0068	ebay	1
24	DCGS0067	ebay	1
25	DCGS0004	HAYNES CAMPER SHOULDER BAG	1
26	DCGS0070	CAMOUFLAGE DOG COLLAR	1
27	DCGS0073	ebay	1
28	m	Manual	1

These are all related to external costs or services and thus are not directly related to sales data, so let's remove them

In [ ]:
```
df = df[df['StockCode'].astype(str).str.match(r'^\d')]
```

Extreme values / outliers

Now that we have gone over a first pass of the data let's reiterate by looking at our statistics again.

In [ ]:
```
df.describe()
```

Out[ ]:

	Quantity	InvoiceDate	UnitPrice	CustomerID
count	532496.000000	532496	532496.000000	399656.000000
mean	9.548175	2011-07-04 12:28:08.476345600	3.284346	15288.781965
min	-80995.000000	2010-12-01 08:26:00	0.000000	12346.000000
25%	1.000000	2011-03-28 11:59:00	1.250000	13959.000000
50%	3.000000	2011-07-19 16:51:00	2.080000	15152.000000
75%	10.000000	2011-10-19 09:00:00	4.130000	16791.000000
max	80995.000000	2011-12-09 12:50:00	649.500000	18287.000000
std	218.892413	NaN	4.510286	1710.791025

There seem to be some extreme Quantity values still so let's look at and assess them.

In [ ]:
```
df_outliers = df[df['UnitPrice'] > 0].sort_values(by='Quantity', key=abs, ascending=False)
df_outliers.head(20)
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Countr
540422	C581484	23843	PAPER CRAFT , LITTLE BIRDIE	-80995	2011-12-09 09:27:00	2.08	16446.0	Unite Kingdor
540421	581483	23843	PAPER CRAFT , LITTLE BIRDIE	80995	2011-12-09 09:15:00	2.08	16446.0	Unite Kingdor
61619	541431	23166	MEDIUM CERAMIC TOP STORAGE JAR	74215	2011-01-18 10:01:00	1.04	12346.0	Unite Kingdor
61624	C541433	23166	MEDIUM CERAMIC TOP STORAGE JAR	-74215	2011-01-18 10:17:00	1.04	12346.0	Unite Kingdor
4287	C536757	84347	ROTATING SILVER ANGELS T-LIGHT HLDR	-9360	2010-12-02 14:23:00	0.03	15838.0	Unite Kingdor
421632	573008	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	4800	2011-10-27 12:26:00	0.21	12901.0	Unite Kingdor
206121	554868	22197	SMALL POPCORN HOLDER	4300	2011-05-27 10:52:00	0.72	13135.0	Unite Kingdor
97432	544612	22053	EMPIRE DESIGN ROSETTE	3906	2011-02-22 10:43:00	0.82	18087.0	Unite Kingdor
270885	560599	18007	ESSENTIAL BALM 3.5g TIN IN ENVELOPE	3186	2011-07-19 17:04:00	0.06	14609.0	Unite Kingdor
52711	540815	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	3114	2011-01-11 12:55:00	2.10	15749.0	Unite Kingdor
160546	550461	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	3114	2011-04-18 13:20:00	2.10	15749.0	Unite Kingdor
160145	C550456	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	-3114	2011-04-18 13:08:00	2.10	15749.0	Unite Kingdor
433788	573995	16014	SMALL CHINESE STYLE SCISSOR	3000	2011-11-02 11:24:00	0.32	16308.0	Unite Kingdor

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Countr
4945	536830	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	2880	2010-12-02 17:38:00	0.18	16754.0	Unite Kingdor
291249	562439	84879	ASSORTED COLOUR BIRD ORNAMENT	2880	2011-08-04 18:06:00	1.45	12931.0	Unite Kingdor
201149	554272	21977	PACK OF 60 PINK PAISLEY CAKE CASES	2700	2011-05-23 13:08:00	0.42	12901.0	Unite Kingdor
80742	543057	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	2592	2011-02-03 10:50:00	0.21	16333.0	Unite Kingdor
421601	573003	23084	RABBIT NIGHT LIGHT	2400	2011-10-27 12:11:00	2.08	14646.0	Netherland
87631	543669	22693	GROW A FLYTRAP OR SUNFLOWER IN TIN	2400	2011-02-11 11:22:00	0.94	16029.0	Unite Kingdor
32671	539101	22693	GROW A FLYTRAP OR SUNFLOWER IN TIN	2400	2010-12-16 10:35:00	0.94	16029.0	Unite Kingdor

The first four of them are fully refunded transactions with extreme Quantity values so we can safely remove them.

In [ ]:
```
df.drop(df_outliers.head(4).index, inplace=True)
```

The data is now clean and will not interfere with any of our analyses with careful filtering.

Exploratory Data Analysis

Filter the data

Now that we have cleaned our data let's create a Revenue column from our data to help with our analysis.

In [ ]:
```
df = df.copy()
df["Revenue"] = df["Quantity"] * df["UnitPrice"]
clean_df = df.copy()
```

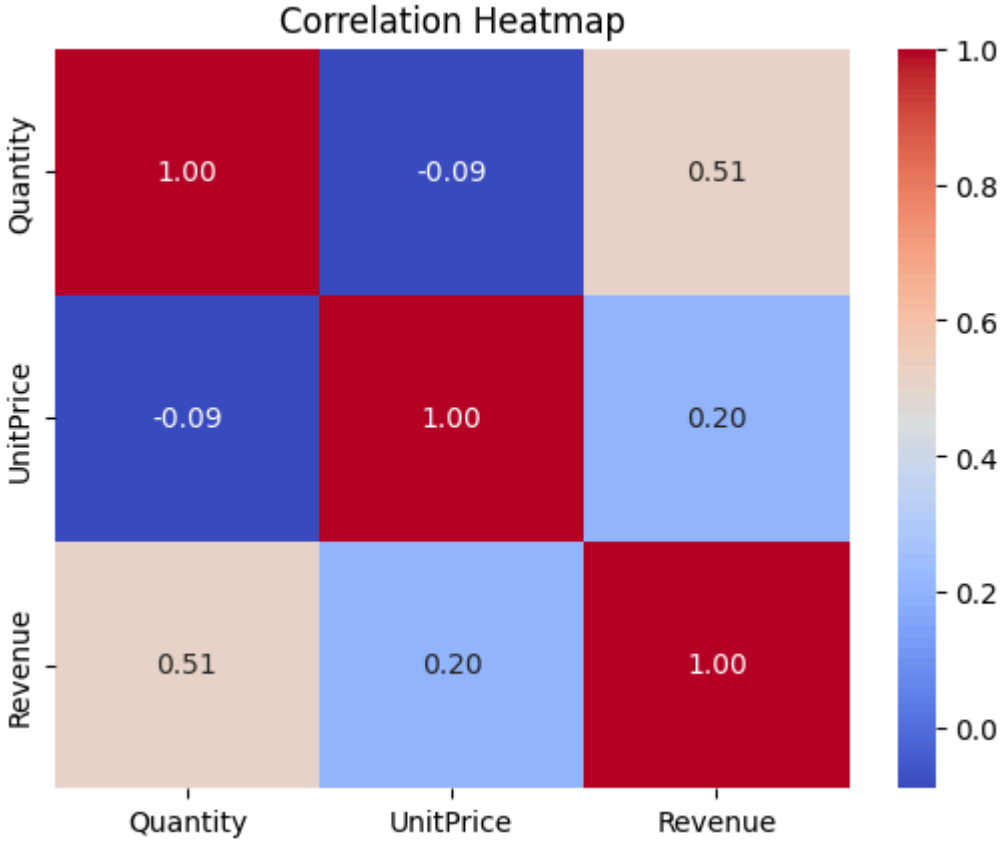
Let's filter for only positive quantities and unitprices for the purpose of our analysis.

In [ ]:
```
filter_positive = df[(df['UnitPrice'] > 0) & (df['Quantity'] > 0)]
```

Numerical features correlations

Let's see if there are any correlations between our numerical columns

In [ ]:
```
corr = filter_positive[["Quantity", "UnitPrice", "Revenue"]].abs().corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



We observe that Quantity and Revenue are moderately positively correlated, which is to be expected.

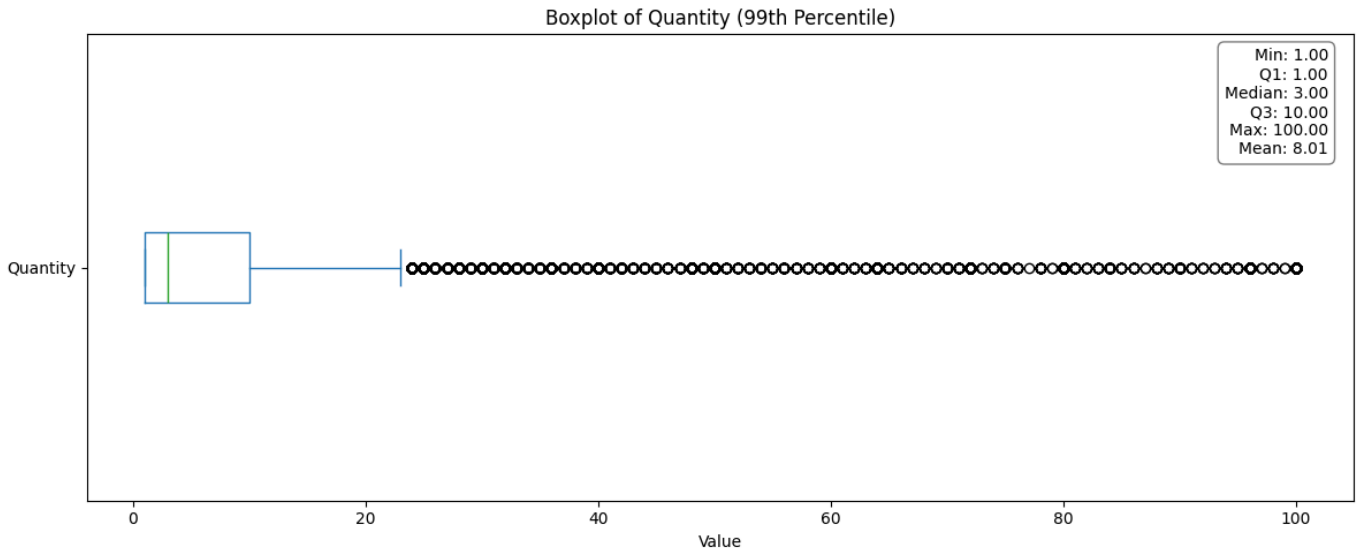
Numerical features visualizations

Let's continue by exploring our numerical values. For the sake of visual clarity and generalization, we are going to focus on the 99th percentile of the corresponding data.

Quantity distribution

In [ ]:
```
q99 = filter_positive['Quantity'].quantile(0.99)

zoomed_df = filter_positive[filter_positive['Quantity'] <= q99]

stats = zoomed_df['Quantity'].describe()
mean_val = zoomed_df['Quantity'].mean()
min_val = stats['min']
q1 = stats['25%']
median = stats['50%']
q3 = stats['75%']
max_val = stats['max']

zoomed_df[['Quantity']].plot.box(vert=False, figsize=(12,5))

plt.title("Boxplot of Quantity (99th Percentile)")
plt.xlabel("Value")
```

```
stats_text = (
    f"Min: {min_val:.2f}\n"
    f"Q1: {q1:.2f}\n"
    f"Median: {median:.2f}\n"
    f"Q3: {q3:.2f}\n"
    f"Max: {max_val:.2f}\n"
    f"Mean: {mean_val:.2f}"
)

plt.gca().text(
    0.98, 0.97, stats_text,
    transform=plt.gca().transAxes,
    fontsize=10,
    verticalalignment='top',
    horizontalalignment='right',
    bbox=dict(boxstyle='round,pad=0.4', facecolor='white', edgecolor='gray')
)

plt.tight_layout()
plt.show()
```

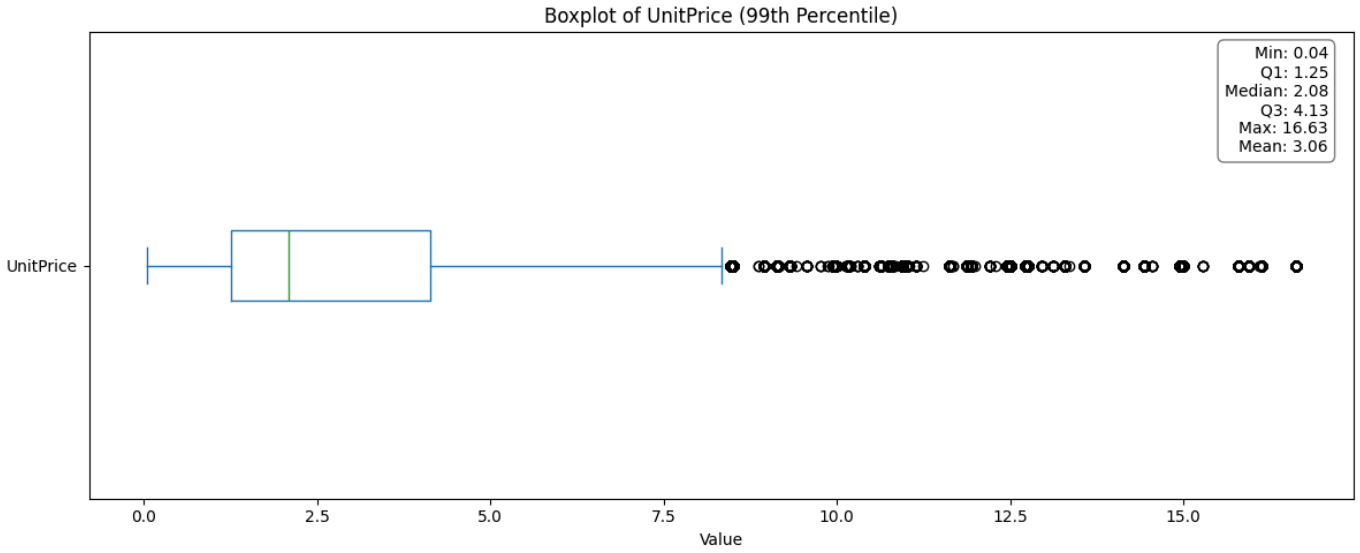


Most transactions include items bought in quantities of 1-10, with a median of 3.

Unitprice distribution

In [ ]:
```
q99 = filter_positive['UnitPrice'].quantile(0.99)

zoomed_df = filter_positive[filter_positive['UnitPrice'] <= q99]

stats = zoomed_df['UnitPrice'].describe()
mean_val = zoomed_df['UnitPrice'].mean()
min_val = stats['min']
q1 = stats['25%']
median = stats['50%']
q3 = stats['75%']
max_val = stats['max']

zoomed_df[['UnitPrice']].plot.box(vert=False, figsize=(12,5))

plt.title("Boxplot of UnitPrice (99th Percentile)")
plt.xlabel("Value")

stats_text = (
    f"Min: {min_val:.2f}\n"
    f"Q1: {q1:.2f}\n"
    f"Median: {median:.2f}\n"
    f"Q3: {q3:.2f}\n"
    f"Max: {max_val:.2f}\n"
```

```
    f"Mean: {mean_val:.2f}"
)

plt.gca().text(
    0.98, 0.97, stats_text,
    transform=plt.gca().transAxes,
    fontsize=10,
    verticalalignment='top',
    horizontalalignment='right',
    bbox=dict(boxstyle='round,pad=0.4', facecolor='white', edgecolor='gray')
)

plt.tight_layout()
plt.show()
```

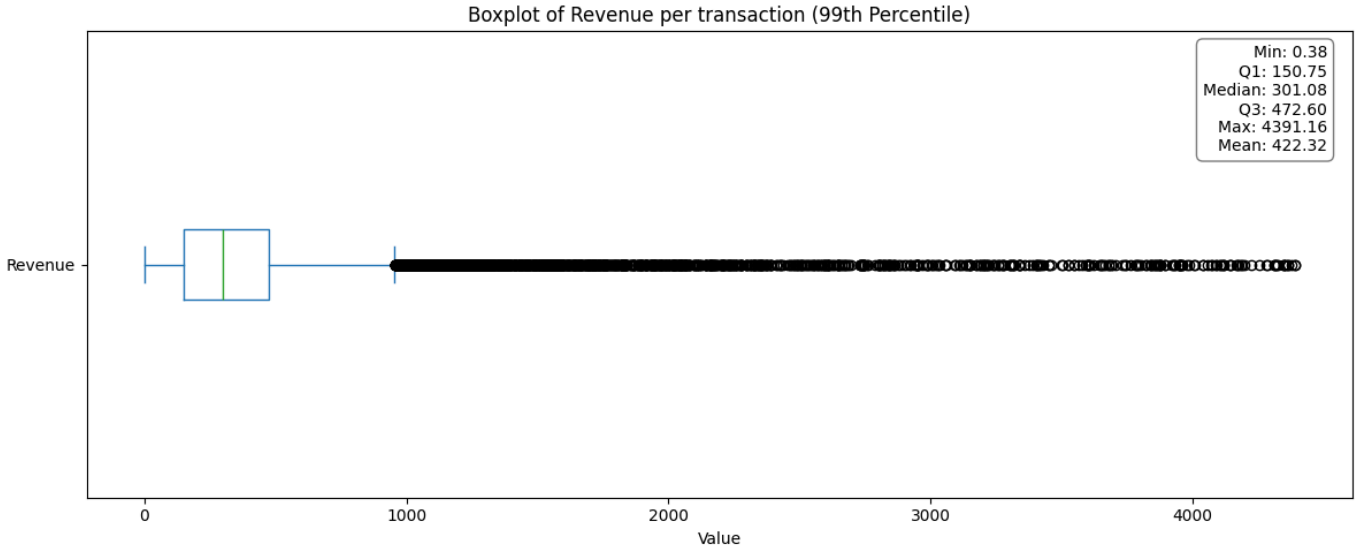


The price per item is mostly in the 1-4 range, with a median of around 2.

Revenue per transaction Distribution

In [ ]:
```
invoice_revenue = filter_positive.groupby("InvoiceNo")["Revenue"].sum()

q99 = invoice_revenue.quantile(0.99)

zoomed_df = invoice_revenue[invoice_revenue <= q99]

stats = zoomed_df .describe()
mean_val = zoomed_df .mean()
min_val = stats['min']
q1 = stats['25%']
median = stats['50%']
q3 = stats['75%']
max_val = stats['max']

zoomed_df.plot.box(vert=False, figsize=(12,5))

plt.title("Boxplot of Revenue per transaction (99th Percentile)")
plt.xlabel("Value")

stats_text = (
    f"Min: {min_val:.2f}\n"
    f"Q1: {q1:.2f}\n"
    f"Median: {median:.2f}\n"
    f"Q3: {q3:.2f}\n"
    f"Max: {max_val:.2f}\n"
    f"Mean: {mean_val:.2f}"
)

plt.gca().text(
```

```
    0.98, 0.97, stats_text,
    transform=plt.gca().transAxes,
    fontsize=10,
    verticalalignment='top',
    horizontalalignment='right',
    bbox=dict(boxstyle='round,pad=0.4', facecolor='white', edgecolor='gray')
)

plt.tight_layout()
plt.show()
```



The median revenue generated from each transaction is around 300.

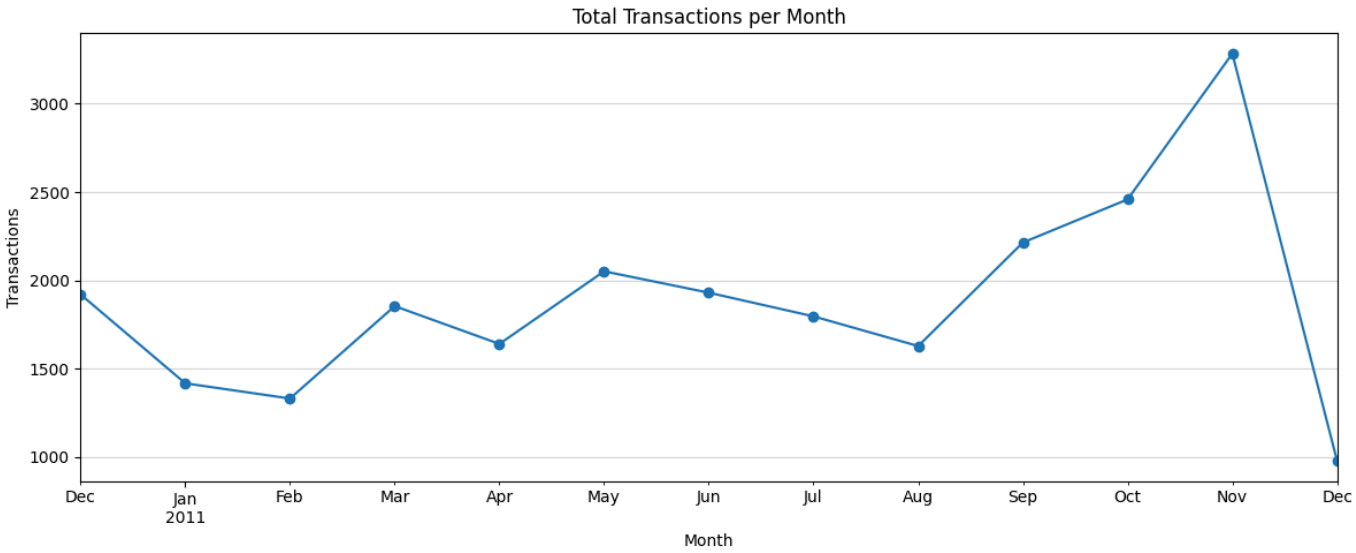
Time series analysis

Now let's move on to some time series sales analysis

In [ ]:
```
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

time_df = df.copy()
time_df['YearMonth'] = df['InvoiceDate'].dt.to_period('M')
time_df['DayOfWeek'] = df['InvoiceDate'].dt.day_name()
time_df['Hour'] = df['InvoiceDate'].dt.hour
```

Transactions per Month

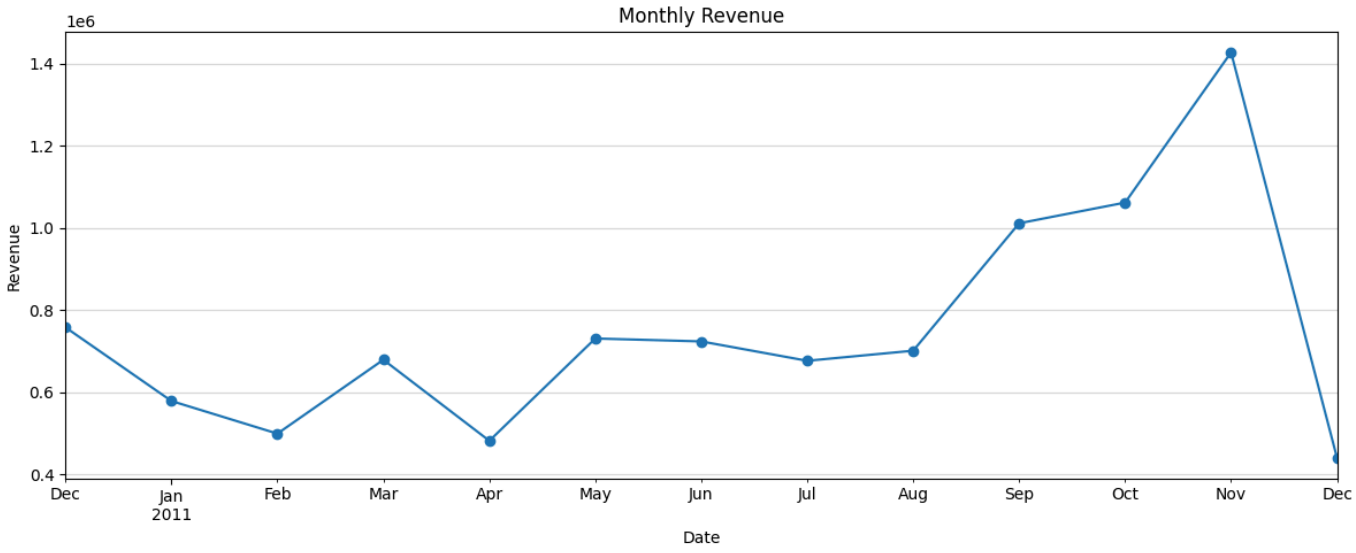
In [ ]:
```
monthly_tx = time_df.groupby('YearMonth')['InvoiceNo'].nunique()
monthly_tx.plot(figsize=(12, 5), marker='o')
plt.title("Total Transactions per Month")
plt.ylabel("Transactions")
plt.xlabel("Month")
plt.grid(True, axis='y', alpha=0.5)
plt.tight_layout()
plt.show()
```



The monthly trend for transactions reveals an increasing trend towards the end of our data. The dropoff in December is explained because we are missing data for the full month.

Revenue per Month

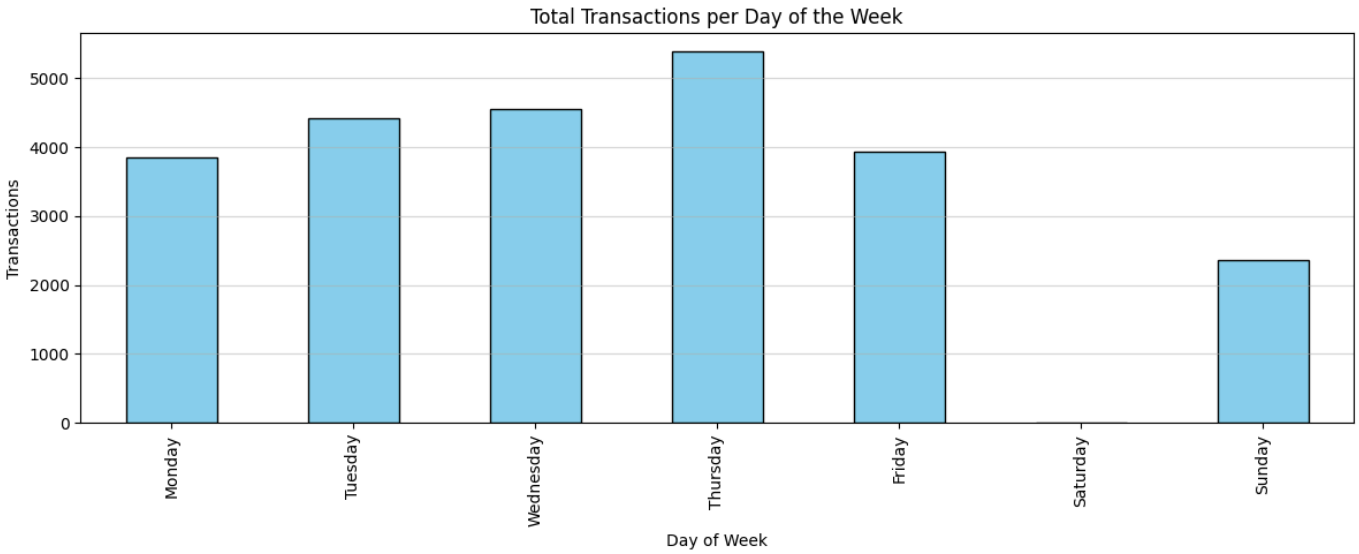
In [ ]:
```
monthly_revenue = time_df.groupby('YearMonth')['Revenue'].sum()
monthly_revenue .plot(figsize=(12, 5), marker='o')
plt.title("Monthly Revenue")
plt.ylabel("Revenue")
plt.xlabel("Date")
plt.grid(True, axis='y', alpha=0.5)
plt.tight_layout()
plt.show()
```



The monthly revenue follows a very similar trend, which is to be expected.

Transactions per day of the week

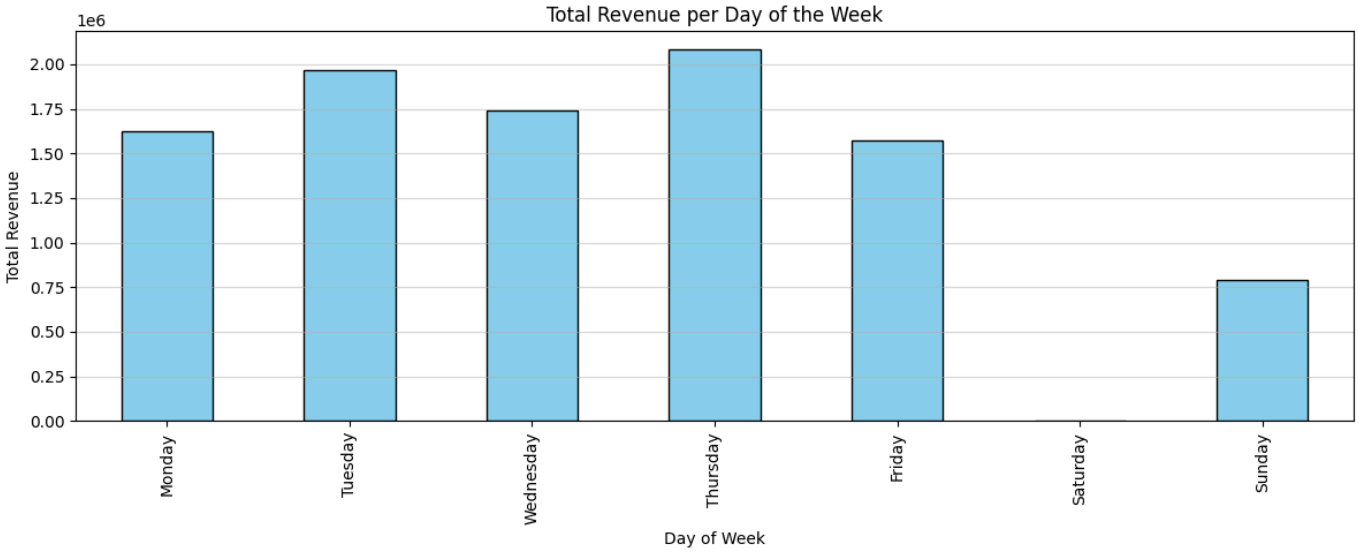
In [ ]:
```
daily_tx = time_df.groupby('DayOfWeek')['InvoiceNo'].nunique().reindex(['Monday', 'Tuesday',

daily_tx.plot(kind='bar', figsize=(12, 5), color='skyblue', edgecolor="black")
plt.title("Total Transactions per Day of the Week")
plt.ylabel("Transactions")
plt.xlabel("Day of Week")
plt.grid(True, axis='y', alpha=0.5)
plt.tight_layout()
plt.show()
```



The transactions per day of the week graph shows that Thursday is the most busy day while Sunday is the least busy one. We can assume that the shop stays closed on Saturdays.

Revenue per day of the week

In [ ]:
```
daily_revenue = time_df.groupby('DayOfWeek')['Revenue'].sum().reindex(['Monday', 'Tuesday', '

daily_revenue.plot(kind='bar', figsize=(12, 5), color='skyblue', edgecolor="black")
plt.title("Total Revenue per Day of the Week")
plt.grid(True, axis='y', alpha=0.5)
plt.ylabel("Total Revenue")
plt.xlabel("Day of Week")

plt.tight_layout()
plt.show()
```

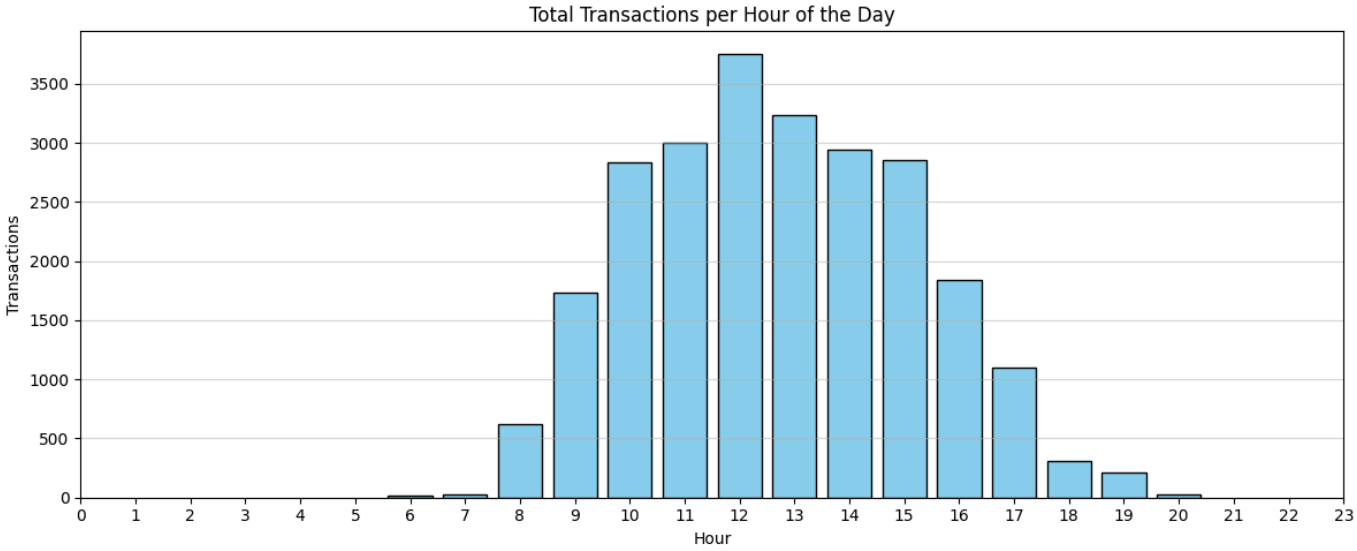


The revenue per day of the week graph closely follows the the transactions trend as expected.

Transactions per hour of the day

In [ ]:
```
hourly_tx = time_df.groupby('Hour')['InvoiceNo'].nunique().reset_index()

plt.figure(figsize=(12,5))
plt.bar(hourly_tx['Hour'], hourly_tx['InvoiceNo'], color='skyblue', edgecolor='black')
plt.title("Total Transactions per Hour of the Day")
plt.ylabel("Transactions")
plt.xlabel("Hour")
plt.grid(True, axis='y', alpha=0.5)
plt.xticks(range(0,24))
```

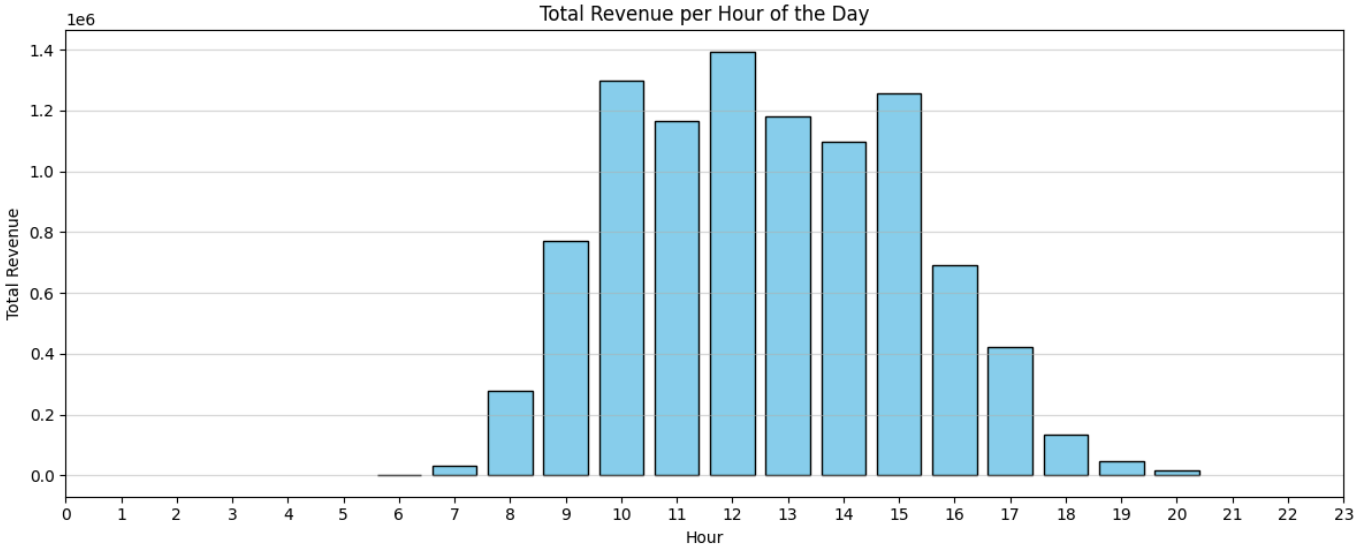
```
plt.tight_layout()
plt.show()
```



The transactions per hour of day graph shows that the most busy hours are those during the middle of the day, dropping off later into the day.

Revenue per hour of day

In [ ]:
```
hourly_revenue = time_df.groupby('Hour')['Revenue'].sum().reset_index()

plt.figure(figsize=(12,5))
plt.bar(hourly_revenue['Hour'], hourly_revenue['Revenue'], color='skyblue', edgecolor='black'
plt.title("Total Revenue per Hour of the Day")
plt.ylabel("Total Revenue")
plt.xlabel("Hour")
plt.grid(True, axis='y', alpha=0.5)
plt.xticks(range(0,24))
plt.tight_layout()
plt.show()
```



Again, the revenue per hour of day follows a very similar trend.

Customer RFM Segmentation

Customer RFM segmentation is a marketing and analytics technique used to understand and categorize customers based on their purchasing behavior. RFM stands for:

Recency (R): How recently a customer made a purchase.

Frequency (F): How often the customer makes purchases.

Monetary Value (M): How much money the customer spends.

This analysis helps personalize marketing campaigns, identifies high-value customers to nurture, detects churn risks and improves resource allocation by targeting the right customer groups.

Data preparation

First I am going to filter the data and calculate the RFM values.

In [ ]:
```
filtered_df = df[(df['Revenue'] > 0) & (df['CustomerID'].notna())].copy()
filtered_df['InvoiceDateOnly'] = filtered_df['InvoiceDate'].dt.date

latest_date = filtered_df['InvoiceDate'].max()
```

In [ ]:
```
RFM = filtered_df.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (latest_date - x.max()).days,
    'InvoiceDateOnly': 'nunique',
    'Revenue': 'sum'
})

RFM.rename(columns={
    'InvoiceDate': 'Recency',
    'InvoiceDateOnly': 'Frequency',
    'Revenue': 'Monetary'
}, inplace=True)

RFM['First_purchase'] = ((latest_date - filtered_df.groupby('CustomerID')['InvoiceDate'].min(
```

In [ ]:
```
RFM = RFM.round(2)
RFM = RFM[RFM['Frequency'] > 0]
RFM = RFM[RFM['Monetary'] >= 0.01]
RFM = RFM.reset_index()
```

In [ ]:
```
RFM['Monetary'] = RFM['Monetary'] / RFM['Frequency']
RFM['Recency'] = RFM['Recency'] // 30
RFM
```

Out[ ]:

	CustomerID	Recency	Frequency	Monetary	First_purchase
0	12347.0	0	7	615.714286	12
1	12348.0	2	4	359.310000	11
2	12349.0	0	1	1457.550000	0
3	12350.0	10	1	294.400000	10
4	12352.0	1	7	197.962857	9
...	...	...	...	...	...
4328	18280.0	9	1	180.600000	9
4329	18281.0	6	1	80.820000	6
4330	18282.0	0	2	89.025000	4
4331	18283.0	0	14	145.684286	11
4332	18287.0	1	3	612.426667	6

4333 rows × 5 columns

Next I am going to look at the distribution of these values and manually remove a few outliers, even if they are valid from a business prespective, in order to help the algorithm generalize better in the next step.

In [ ]:
```
RFM.describe()
```

Out[ ]:

	CustomerID	Recency	Frequency	Monetary	First_purchase
count	4333.000000	4333.000000	4333.000000	4333.000000	4333.000000
mean	15299.933303	2.621048	3.848604	414.972603	6.953381
std	1721.608094	3.322458	5.926697	858.993794	3.972743
min	12347.000000	0.000000	1.000000	2.900000	0.000000
25%	13813.000000	0.000000	1.000000	181.898333	3.000000
50%	15298.000000	1.000000	2.000000	300.360000	8.000000
75%	16779.000000	4.000000	4.000000	443.690000	10.000000
max	18287.000000	12.000000	130.000000	39916.500000	12.000000

In [ ]:
```
RFM[['CustomerID', 'Monetary']].sort_values(by='Monetary', ascending=False).head(20)
```

Out[ ]:

	CustomerID	Monetary
2010	15098.0	39916.500000
2501	15749.0	22267.150000
2675	16000.0	12393.700000
4196	18102.0	9986.819231
195	12590.0	9341.260000
54	12415.0	8304.302000
3724	17450.0	7199.658889
9	12357.0	6207.670000
1688	14646.0	6203.067111
277	12688.0	4873.810000
328	12752.0	4366.780000
4309	18251.0	4314.720000
151	12536.0	4279.710000
4223	18139.0	4219.170000
1283	14088.0	4207.650833
3173	16684.0	4165.847500
26	12378.0	4008.620000
72	12435.0	3914.945000
2086	15195.0	3861.000000
49	12409.0	3690.890000

In [ ]:
```
RFM[['CustomerID', 'Frequency']].sort_values(by='Frequency', ascending=False).head(20)
```

Out[ ]:

	CustomerID	Frequency
1878	14911.0	130
325	12748.0	113
4006	17841.0	112
2175	15311.0	90
1660	14606.0	88
480	12971.0	71
561	13089.0	66
1601	14527.0	54
1068	13798.0	53
2987	16422.0	48
1688	14646.0	45
1332	14156.0	43
1963	15039.0	42
794	13408.0	41
2082	15189.0	38
3988	17811.0	38
2699	16029.0	38
834	13468.0	36
4097	17961.0	35
995	13694.0	35

In [ ]:
```
RFM.drop(RFM[['CustomerID', 'Monetary']].sort_values(by='Monetary', ascending=False).head(10)
RFM.drop(RFM[['CustomerID', 'Frequency']].sort_values(by='Frequency', ascending=False).head(5
RFM.reset_index(drop=True, inplace=True)
```

I have removed 10 customers based on their monetary value and 5 based on their frequency value. These are less than 0.01% of our customers but will go a long way with helping the segmentation process.

Clustering

I am going to be using clustering with K-means for the segmentation process.

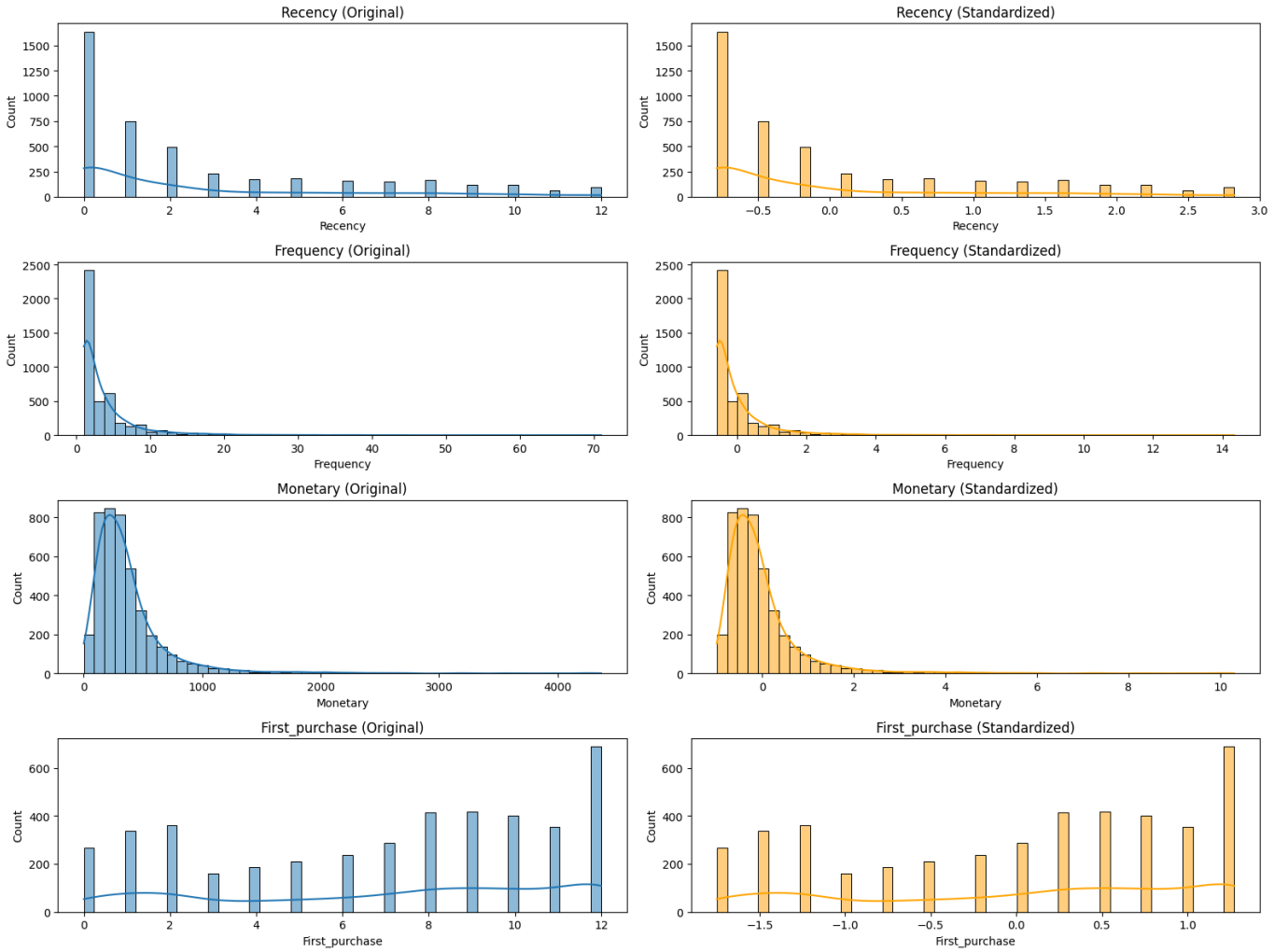
In [ ]:
```
X = RFM[['Recency', 'Frequency', 'Monetary', 'First_purchase']]
```

First I need to standardize our data, let's also visualize it.

In [ ]:
```
X_scaled = StandardScaler().fit_transform(X.to_numpy())
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
```

In [ ]:
```
fig, axes = plt.subplots(4, 2, figsize=(16, 12))

for i, col in enumerate(['Recency', 'Frequency', 'Monetary', 'First_purchase']):
    # Original
```

```
    sns.histplot(X[col], bins=50, ax=axes[i,0], kde=True)
    axes[i,0].set_title(f"{col} (Original)")

    # Standardized
    sns.histplot(X_scaled_df[col], bins=50, ax=axes[i,1], kde=True, color='orange')
    axes[i,1].set_title(f"{col} (Standardized)")

plt.tight_layout()
plt.show()
```

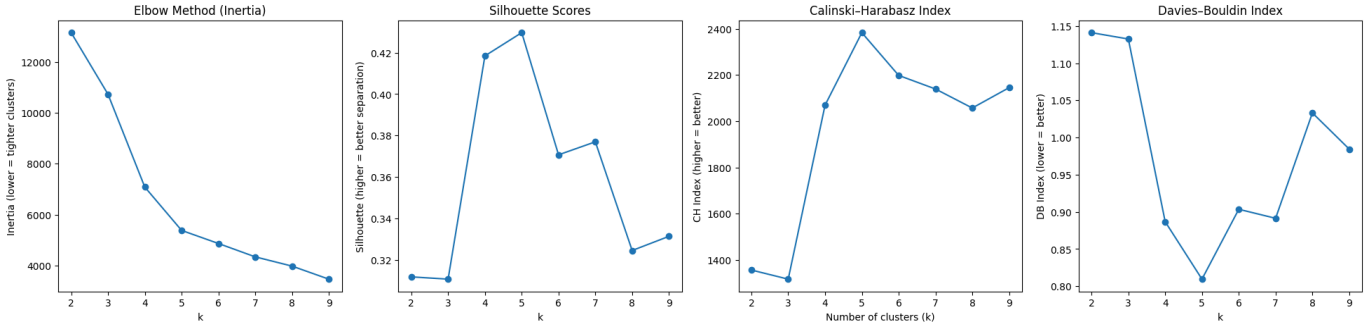


K-means needs a predetermined number of clusters (k)

There are several metrics which can suggest an appropriate k-value, some of which I am going to be using, but the final and most crucial criterion should always be for the clusters to be interpretable and hold meaning and value from a business prespective.

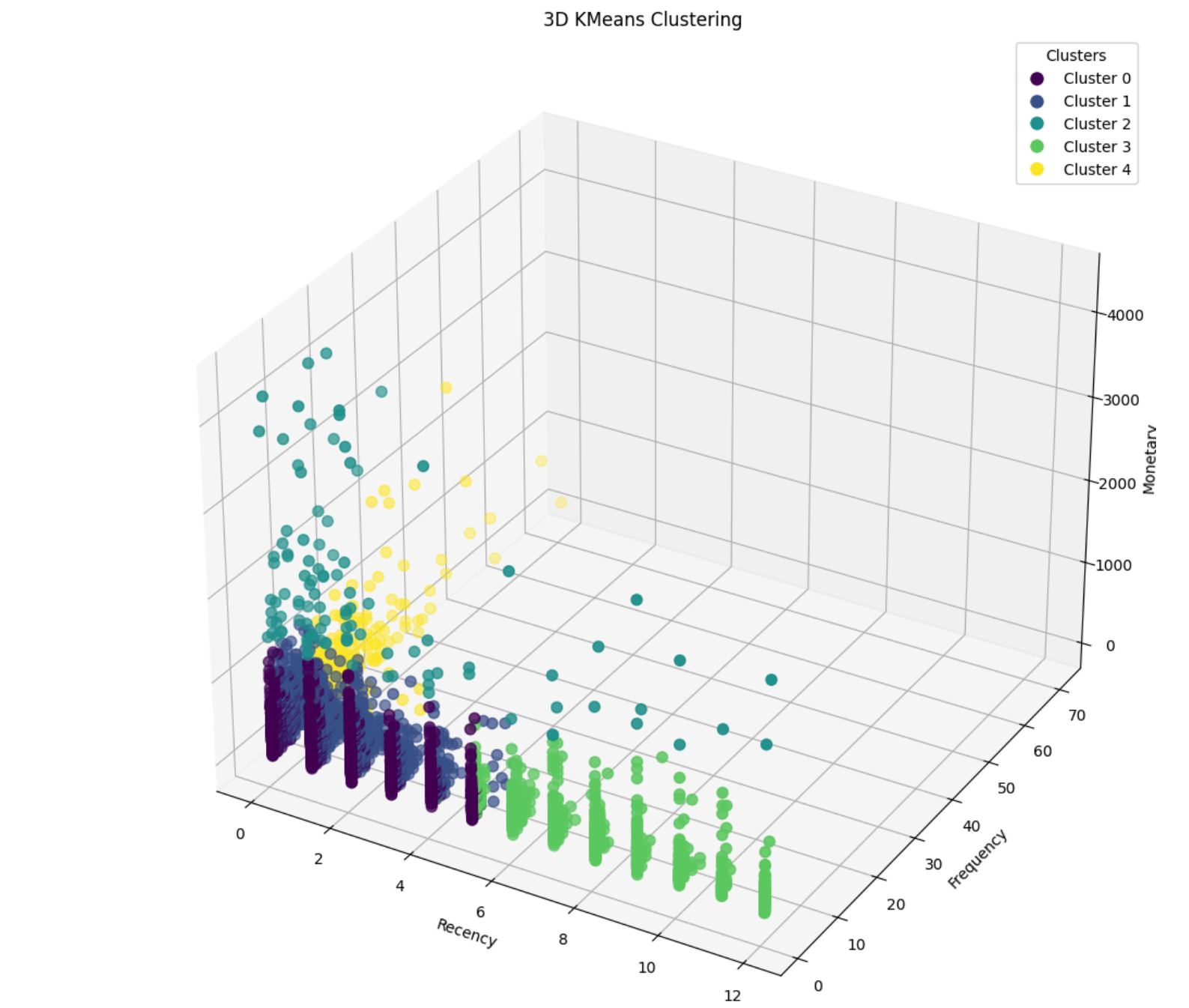
In [ ]:
```
inertias, sil_scores, ch_scores, db_scores = [], [], [], []
K = range(2, 10)

for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init='auto').fit(X_scaled)
    labels = km.labels_

    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))
    ch_scores.append(calinski_harabasz_score(X_scaled, labels))
    db_scores.append(davies_bouldin_score(X_scaled, labels))

fig, axes = plt.subplots(1, 4, figsize=(24,5))

#Plot Elbow curve (Inertia)
axes[0].plot(K, inertias, 'o-')
axes[0].set_title(f"Elbow Method (Inertia)")
axes[0].set_xlabel("k")
```

```
axes[0].set_ylabel("Inertia (lower = tighter clusters)")

#Plot Silhouette Scores
axes[1].plot(K, sil_scores, 'o-')
axes[1].set_title(f"Silhouette Scores")
axes[1].set_xlabel("k")
axes[1].set_ylabel("Silhouette (higher = better separation)")

#Plot Calinski-Harabasz index
axes[2].plot(K, ch_scores, 'o-')
axes[2].set_title(f"Calinski-Harabasz Index")
axes[2].set_xlabel("Number of clusters (k)")
axes[2].set_ylabel("CH Index (higher = better)")

#Plot Davies-Bouldin index
axes[3].plot(K, db_scores, 'o-')
axes[3].set_title(f"Davies-Bouldin Index")
axes[3].set_xlabel("k")
axes[3].set_ylabel("DB Index (lower = better)")

plt.show()
```



The indexes point towards k=5, but the final decision will have to take place after looking at the resulting clusters.

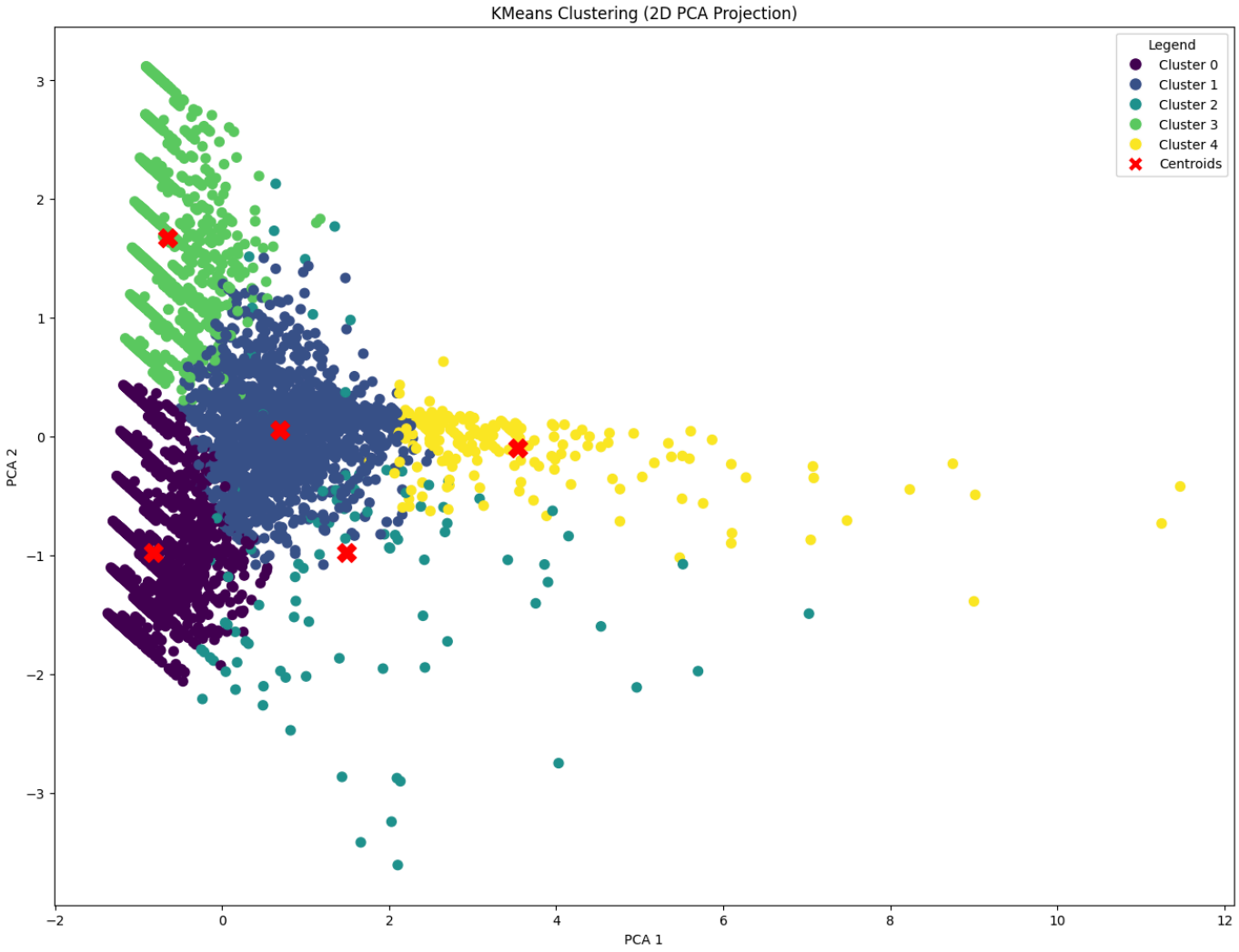
In [ ]:
```
km = KMeans(n_clusters=5, random_state=42, n_init='auto')
km.fit(X_scaled)
labels = km.labels_

fig = plt.figure(figsize=(16,12))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X.to_numpy()[:, 0],
    X.to_numpy()[:, 1],
    X.to_numpy()[:, 2],
    c=labels, cmap='viridis', s=50
)

ax.set_xlabel('Recency')
ax.set_ylabel('Frequency')
ax.set_zlabel('Monetary')
ax.set_title('3D KMeans Clustering')

legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=f'Cluster {i}',
           markerfacecolor=scatter.cmap(scatter.norm(i)), markersize=10)
    for i in range(km.n_clusters)
]

ax.legend(handles=legend_elements, title="Clusters")
plt.show()
```



In [ ]:

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(16,12))

scatter = plt.scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=km.labels_, cmap='viridis', s=50
)

centroids_pca = pca.transform(km.cluster_centers_)
plt.scatter(
    centroids_pca[:, 0], centroids_pca[:, 1],
    c='red', marker='X', s=200, label='Centroids'
)

plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('KMeans Clustering (2D PCA Projection)')

legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=f'Cluster {i}',
           markerfacecolor=scatter.cmap(scatter.norm(i)), markersize=10)
```

```
    for i in range(km.n_clusters)
]

legend_elements.append(
    Line2D([0], [0], marker='X', color='w', label='Centroids',
           markerfacecolor='red', markersize=12)
)

plt.legend(handles=legend_elements, title="Legend")
plt.show()
```



Interpretation

Now we can calculate statistics such as the mean and median RFM, revenue percentage and customer counts, for each individual cluster.

In [ ]:
```
RFM['Cluster'] = labels

RFM['Total_Spent'] = RFM['Monetary'] * RFM['Frequency']

cluster_summary = (
    RFM
    .groupby("Cluster", observed=False)[["Recency", "Frequency", "Monetary", "First_purchase"
    .agg(["mean", "median"])
    .round(2)
)

cluster_summary.columns = [
    f"{metric}_{stat}" for metric, stat in cluster_summary.columns
]
cluster_summary = cluster_summary.reset_index()
```

```
Total_revenue = RFM['Total_Spent'].sum()

Monetary_percentage = (RFM.groupby('Cluster', observed=False)['Total_Spent'].sum() / Total_re

cluster_summary['Revenue_percentage'] = Monetary_percentage
cluster_summary['Customer_count'] = RFM['Cluster'].groupby(RFM['Cluster'], observed=False).co
cluster_summary['Customer_percentage'] = (cluster_summary['Customer_count'] / cluster_summary

cluster_summary
```

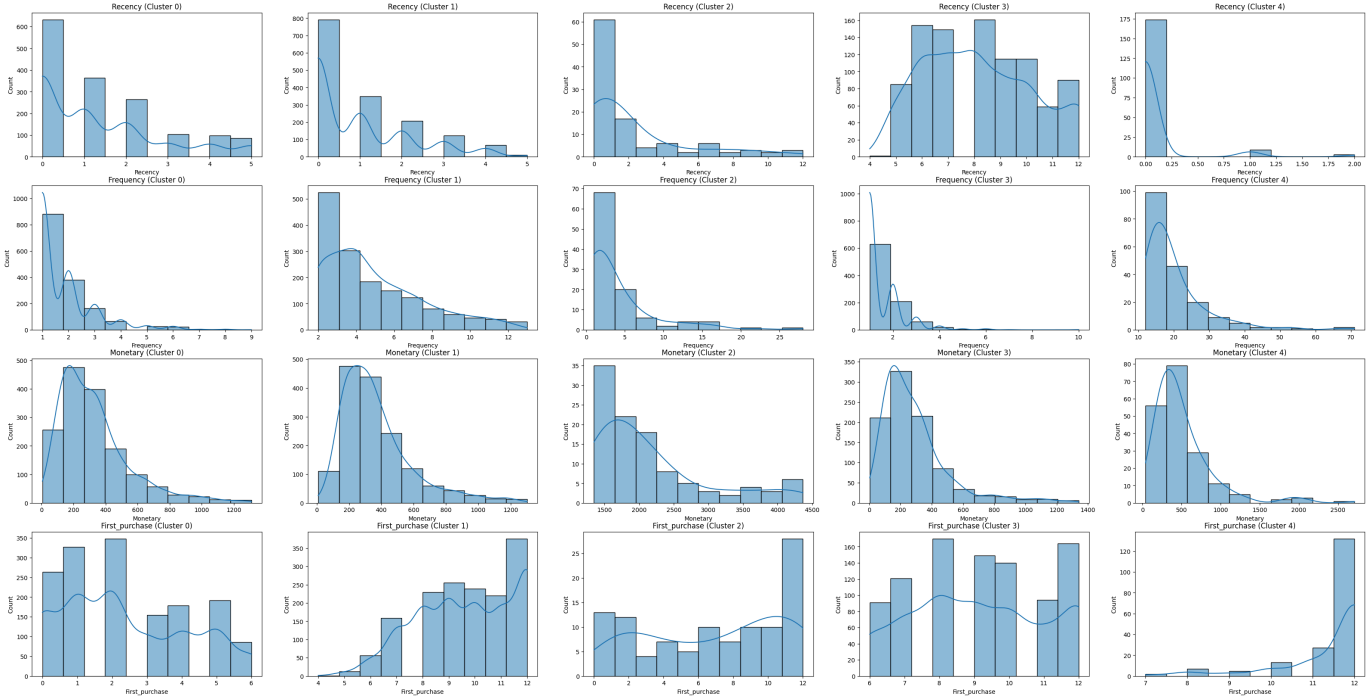
Out[ ]:

	Cluster	Recency_mean	Recency_median	Frequency_mean	Frequency_median	Monetary_mean	Mo
0	0	1.31	1.0	1.77	1.0	327.01	
1	1	0.94	0.0	4.99	4.0	368.65	
2	2	2.28	1.0	4.08	2.0	2157.83	
3	3	8.18	8.0	1.48	1.0	289.12	
4	4	0.08	0.0	20.21	17.0	507.49	

I am satisfied with the resulting clusters but let's also look at the RFM distribution before interpreting each cluster.

In [ ]:

```
fig, axes = plt.subplots(4, 5, figsize=(40, 20))

for i, col in enumerate(['Recency', 'Frequency', 'Monetary', 'First_purchase']):
    sns.histplot(RFM[RFM['Cluster'] == 0][col], bins=10, ax=axes[i,0], kde=True)
    axes[i,0].set_title(f"{col} (Cluster 0)")

    sns.histplot(RFM[RFM['Cluster'] == 1][col], bins=10, ax=axes[i,1], kde=True)
    axes[i,1].set_title(f"{col} (Cluster 1)")

    sns.histplot(RFM[RFM['Cluster'] == 2][col], bins=10, ax=axes[i,2], kde=True)
    axes[i,2].set_title(f"{col} (Cluster 2)")

    sns.histplot(RFM[RFM['Cluster'] == 3][col], bins=10, ax=axes[i,3], kde=True)
    axes[i,3].set_title(f"{col} (Cluster 3)")

    sns.histplot(RFM[RFM['Cluster'] == 4][col], bins=10, ax=axes[i,4], kde=True)
    axes[i,4].set_title(f"{col} (Cluster 4)")
```



Cluster 0: This is the largest and most varied cluster. It consists of shoppers of the last 6 months which may be new, regular, or occasional. They have shopped with varying frequencies and low-medium spending. They represent 35% of the customers and they contribute 12.5% of the revenue. It would be helpful to seperate those that are new and likely to come back, from those that are not and spend very little. You want to attract the former to come back more often and preserve them as regular customers while not wasting resources on the latter.

Cluster 1: These are your regular customers. They have been shopping for a while and they keep coming back. They represent 35% of your customers and have contributed 40% of your revenue. Preserving them should not be difficult.

Cluster 2: These are occasional, very high-spending customers. They might be organizational and not individual customers. They are the smallest fraction of your customers, they shop in varying frequencies but in large quantities. Some of them have not come back in a while. You want to keep in touch with them to ensure they come back and possibly offer them large quantity deals.

Cluster 3: This is a significant portion of your customers (20%) who have shopped once or twice a while ago but have not recently come back. They have contributed the smallest part of your revenue (5.5%) and the resources required to bring them back are likely not worth the return.

Cluster 4: These are your loyal customers. They have shopped recently, very frequently and in good amounts. While they represent only 4.5% of your customers, they generate 28% of your revenue. You want to keep them happy at all costs.

Subclustering

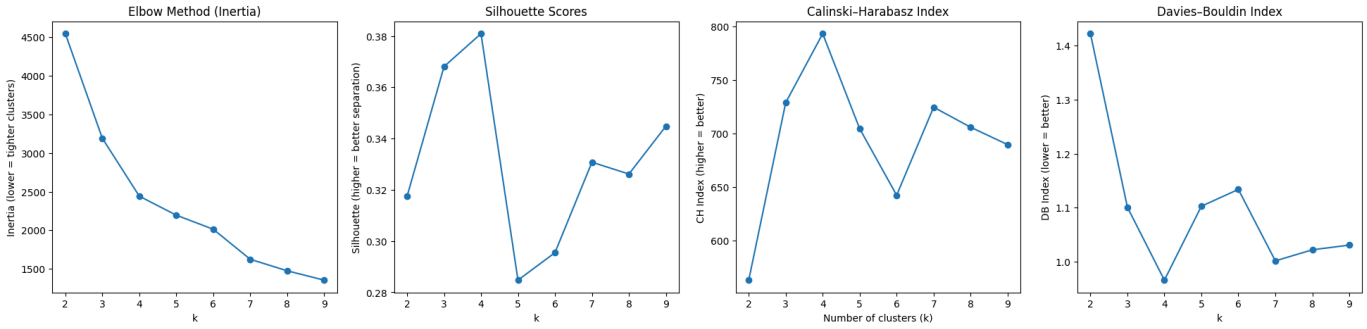
I am going to run subclustering for Cluster 0 specifically and see if we can split it into more defined parts.

In [ ]:
```
Cluster = RFM[RFM['Cluster'] == 0][['Recency', 'Frequency', 'Monetary', 'First_purchase']].co
```

In [ ]:
```
X_scaled = StandardScaler().fit_transform(Cluster)
```

In [ ]:
```
inertias, sil_scores, ch_scores, db_scores = [], [], [], []
K = range(2, 10)
```

```
for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init='auto').fit(X_scaled)
    labels = km.labels_

    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))
    ch_scores.append(calinski_harabasz_score(X_scaled, labels))
    db_scores.append(davies_bouldin_score(X_scaled, labels))

fig, axes = plt.subplots(1, 4, figsize=(24,5))

axes[0].plot(K, inertias, 'o-')
axes[0].set_title(f"Elbow Method (Inertia)")
axes[0].set_xlabel("k")
axes[0].set_ylabel("Inertia (lower = tighter clusters)")

axes[1].plot(K, sil_scores, 'o-')
axes[1].set_title(f"Silhouette Scores")
axes[1].set_xlabel("k")
axes[1].set_ylabel("Silhouette (higher = better separation)")

axes[2].plot(K, ch_scores, 'o-')
axes[2].set_title(f"Calinski-Harabasz Index")
axes[2].set_xlabel("Number of clusters (k)")
axes[2].set_ylabel("CH Index (higher = better)")

axes[3].plot(K, db_scores, 'o-')
axes[3].set_title(f"Davies-Bouldin Index")
axes[3].set_xlabel("k")
axes[3].set_ylabel("DB Index (lower = better)")

plt.show()
```

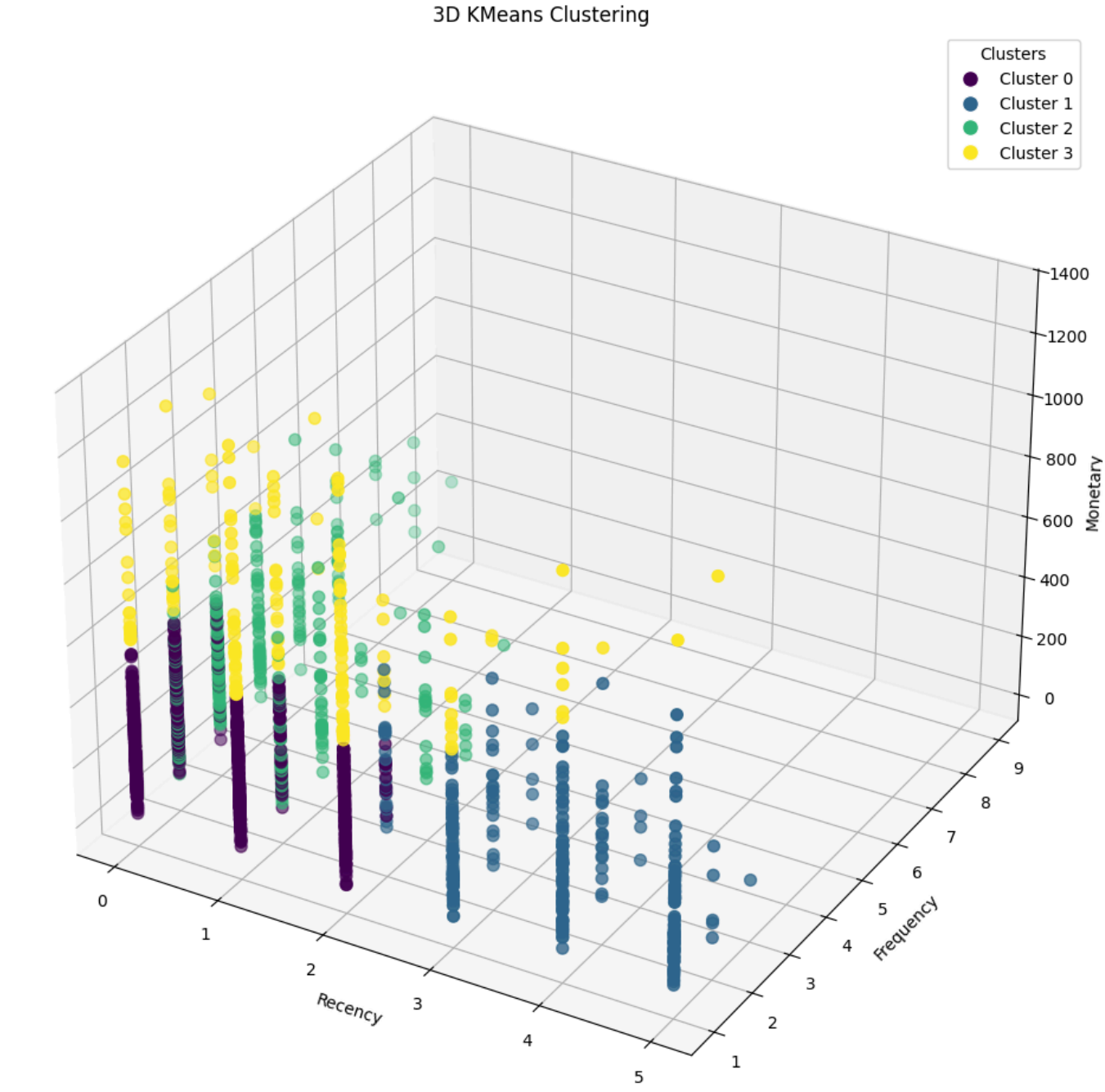


Again, the indexes point towards k=4, but the final decision will have to take place after looking at the resulting clusters.

In [ ]:

```
km = KMeans(n_clusters=4, random_state=42, n_init='auto')
km.fit(X_scaled)
labels = km.labels_

fig = plt.figure(figsize=(16,12))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    Cluster.to_numpy()[:, 0],
    Cluster.to_numpy()[:, 1],
    Cluster.to_numpy()[:, 2],
    c=labels, cmap='viridis', s=50
)

ax.set_xlabel('Recency')
ax.set_ylabel('Frequency')
ax.set_zlabel('Monetary')
ax.set_title('3D KMeans Clustering')
```

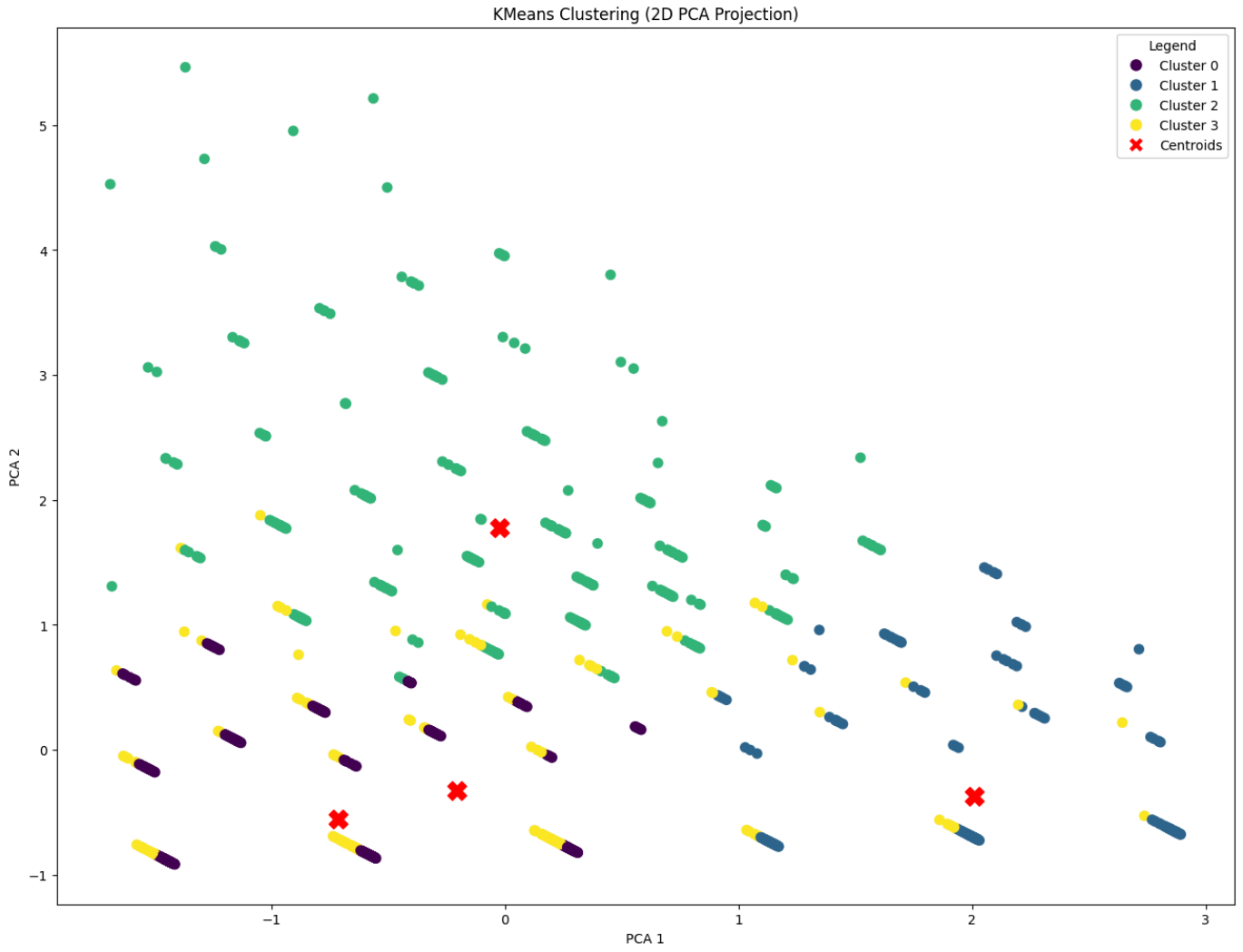
```
legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=f'Cluster {i}',
           markerfacecolor=scatter.cmap(scatter.norm(i)), markersize=10)
    for i in range(km.n_clusters)
]

ax.legend(handles=legend_elements, title="Clusters")
plt.show()
```



In [ ]:

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(16,12))

scatter = plt.scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=km.labels_, cmap='viridis', s=50
)

centroids_pca = pca.transform(km.cluster_centers_)
plt.scatter(
    centroids_pca[:, 0], centroids_pca[:, 1],
    c='red', marker='X', s=200, label='Centroids'
```

```
)

plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('KMeans Clustering (2D PCA Projection)')

legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=f'Cluster {i}',
           markerfacecolor=scatter.cmap(scatter.norm(i)), markersize=10)
    for i in range(km.n_clusters)
]

legend_elements.append(
    Line2D([0], [0], marker='X', color='w', label='Centroids',
           markerfacecolor='red', markersize=12)
)

plt.legend(handles=legend_elements, title="Legend")
plt.show()
```



Once again let's calculate RFM statistics, revenue percentage and customer counts, for each individual subcluster.

In [ ]:
```
Cluster['Cluster'] = labels

Cluster['Total_Spent'] = Cluster['Monetary'] * Cluster['Frequency']

cluster_summary = (
    Cluster
    .groupby("Cluster", observed=False)[["Recency", "Frequency", "Monetary", 'First_purchase'
    .agg(["mean", "median"])
    .round(2)
)
```

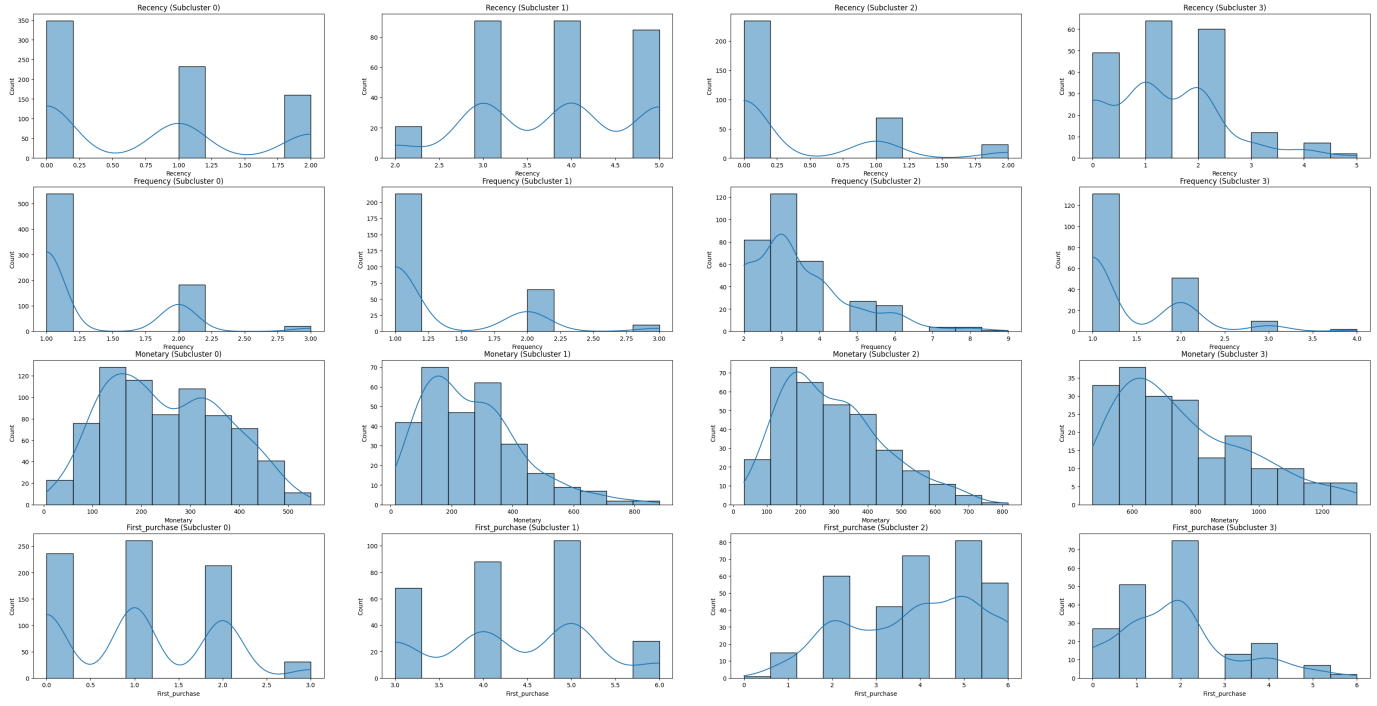
```
cluster_summary.columns = [
    f"{metric}_{stat}" for metric, stat in cluster_summary.columns
]
cluster_summary = cluster_summary.reset_index()

Total_revenue = Cluster['Total_Spent'].sum()

Monetary_percentage = (Cluster.groupby('Cluster', observed=False)['Total_Spent'].sum() / Tota


cluster_summary['Revenue_percentage'] = Monetary_percentage
cluster_summary['Customer_count'] = Cluster['Cluster'].groupby(Cluster['Cluster'], observed=F
cluster_summary['Customer_percentage'] = (cluster_summary['Customer_count'] / cluster_summary

cluster_summary
```

Out[ ]:

	Cluster	Recency_mean	Recency_median	Frequency_mean	Frequency_median	Monetary_mean	Mo
0	0	0.74	1.0	1.30	1.0	250.82	
1	1	3.83	4.0	1.30	1.0	265.82	
2	2	0.35	0.0	3.45	3.0	296.93	
3	3	1.33	1.0	1.40	1.0	759.58	

In [ ]:

```
fig, axes = plt.subplots(4, 4, figsize=(40, 20))

for i, col in enumerate(['Recency', 'Frequency', 'Monetary', 'First_purchase']):
    sns.histplot(Cluster[Cluster['Cluster'] == 0][col], bins=10, ax=axes[i,0], kde=True)
    axes[i,0].set_title(f"{col} (Subcluster 0)")

    sns.histplot(Cluster[Cluster['Cluster'] == 1][col], bins=10, ax=axes[i,1], kde=True)
    axes[i,1].set_title(f"{col} (Subcluster 1)")

    sns.histplot(Cluster[Cluster['Cluster'] == 2][col], bins=10, ax=axes[i,2], kde=True)
    axes[i,2].set_title(f"{col} (Subcluster 2)")

    sns.histplot(Cluster[Cluster['Cluster'] == 3][col], bins=10, ax=axes[i,3], kde=True)
    axes[i,3].set_title(f"{col} (Subcluster 3)")
```



Looks like there are some clearer patterns forming in each subcluster.

Subcluster 0: These are your new customers. They have mostly shopped once in the last 3 months as their first transaction. Their monetary spending is evenly distributed.

Subcluster 1: These are older customers who are unlikely to come back, very similar to the earlier cluster 3. Their spending is also on the low-end. I am going to be merging them. -> 3

Subcluster 2: These are very similar to your regular customers. They have shopped frequently and they keep coming back. Their spending is low-medium. I am going to be merging them with the earlier regular customers. -> 1

Subcluster 3: These are new or returning customers with the characteristic of high spending. They are not quite a regular or a loyal customer yet and their spending is not quite on the level of organizational customers either. While they are a very small fraction, it is probably best to seperate them as a category as they are probably very good promotion targets. -> 5

Results

Merge the subclusters with the original clusters

In [ ]:
```
Cluster[Cluster['Cluster'] == 3] = 5
Cluster[Cluster['Cluster'] == 1] = 3
Cluster[Cluster['Cluster'] == 2] = 1
```

In [ ]:
```
RFM['Subcluster'] = RFM['Cluster']
```

In [ ]:
```
RFM.loc[
    RFM['Cluster'] == 0,
    'Subcluster'
] = Cluster['Cluster']
```

In [ ]:
```
RFM.loc[
    RFM['Cluster'] == 0,
    'Cluster'
] = RFM['Subcluster']
```

In [ ]:
```
RFM.drop(columns='Subcluster', inplace=True)
```

In [ ]:
```
Plot_metrics = RFM.copy()
```

Let's rename the final clusters for clarity.

In [ ]:
```
cluster_labels = {
    0: "New",
    1: "Regular",
    2: "Organizational",
    3: "Low Value",
    4: "Loyal",
    5: "High Spender"
}

RFM['Cluster'] = RFM['Cluster'].map(cluster_labels)
```

In [ ]:
```
RFM['Total_Spent'] = RFM['Monetary'] * RFM['Frequency']

cluster_summary = (
    RFM
    .groupby("Cluster", observed=False)[["Recency", "Frequency", "Monetary", "First_purchase"
    .agg(["mean", "median"])
```

```
    .round(2)
)

cluster_summary.columns = [
    f"{metric}_{stat}" for metric, stat in cluster_summary.columns
]
cluster_summary = cluster_summary.reset_index()

Total_revenue = RFM['Total_Spent'].sum()

Monetary_percentage = (RFM.groupby('Cluster', observed=False)['Total_Spent'].sum() / Total_re


cluster_summary['Revenue_percentage'] = Monetary_percentage
cluster_summary['Customer_count'] = RFM['Cluster'].groupby(RFM['Cluster'], observed=False).co
cluster_summary['Customer_percentage'] = (cluster_summary['Customer_count'] / cluster_summary

cluster_summary
```

Out[ ]:

	Cluster	Recency_mean	Recency_median	Frequency_mean	Frequency_median	Monetary_mea
0	High Spender	1.33	1.0	1.40	1.0	759.5
1	Low Value	7.15	7.0	1.43	1.0	283.6
2	Loyal	0.08	0.0	20.21	17.0	507.4
3	New	0.74	1.0	1.30	1.0	250.8
4	Organizational	2.28	1.0	4.08	2.0	2157.8
5	Regular	0.83	0.0	4.72	4.0	356.1

Low Value: This is a significant portion of your customers (28%) who have shopped once or twice and have not come back in the last 6 months. They have contributed the smallest part of your revenue (7%) and the resources required to bring them back are likely not worth the return.

New: These are your new customers. They have mostly shopped once in the last 3 months as their first transaction. Their monetary spending is low-medium.

Regular: These are your regular customers. They have been shopping for a while and they keep coming back. They represent 43% of your customers and have contributed 45% of your revenue. Your goal is to preserve them.

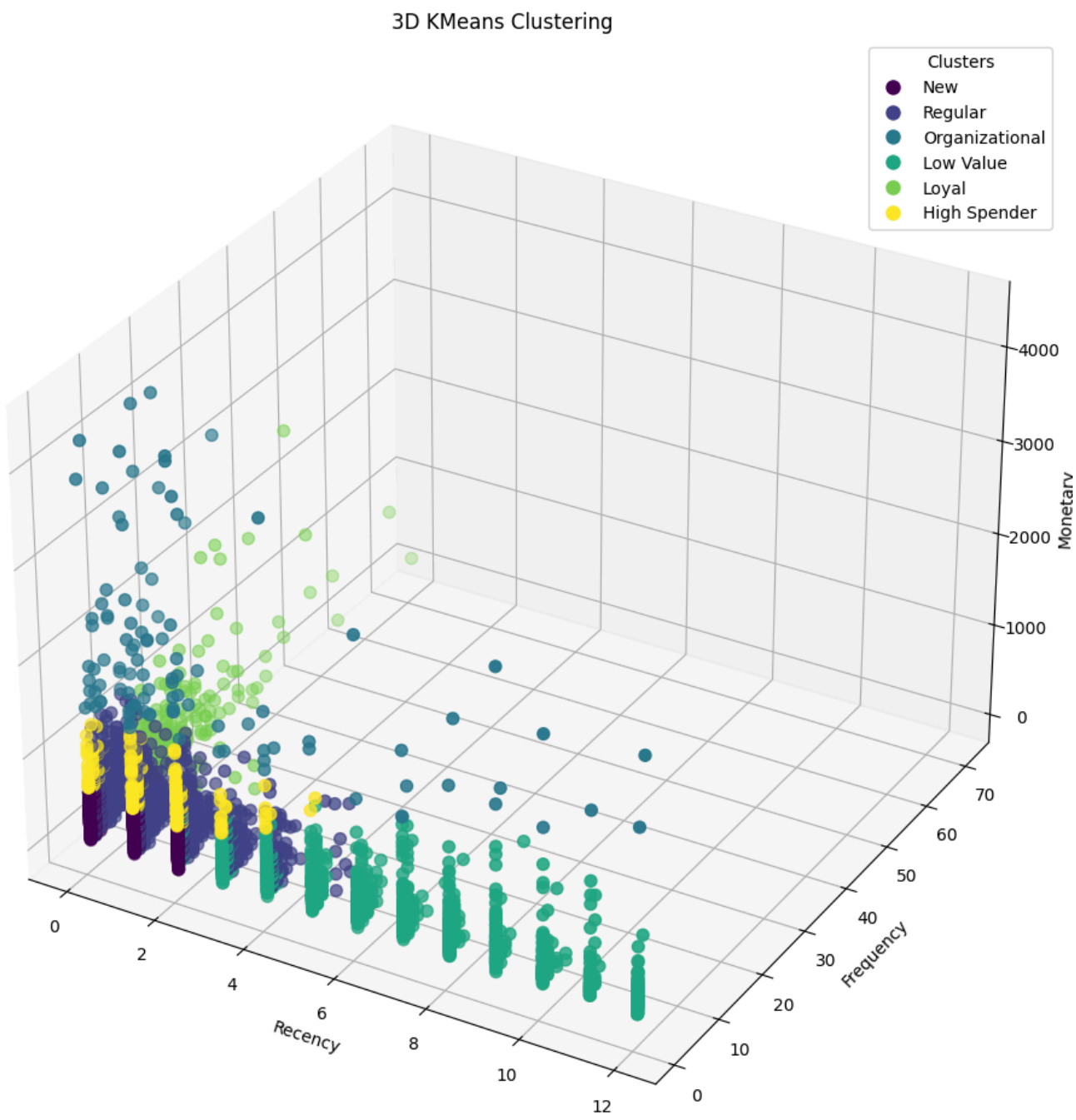
Organizational: These are occasional, very high-spending customers. They are likely organizational and not individual customers. They are the smallest fraction of your customers (2.5%), they shop in varying frequencies but in large quantities. Some of them have not come back in a while. You want to keep in touch with them to ensure they come back and possibly offer them large quantity deals.

Loyal: These are your loyal customers. They have shopped recently, very frequently and in good amounts. While they represent only 4.5% of your customers, they generate 28% of your revenue. You want to keep them happy at all costs.

High Spender: These are new or returning customers with the characteristic of high spending. They are not quite a regular or a loyal customer yet and their spending is not quite on the level of organizational customers either. While they are a very small fraction (4.5%), it is probably best to seperate them as a category as they are probably very good promotion targets.

In [ ]:

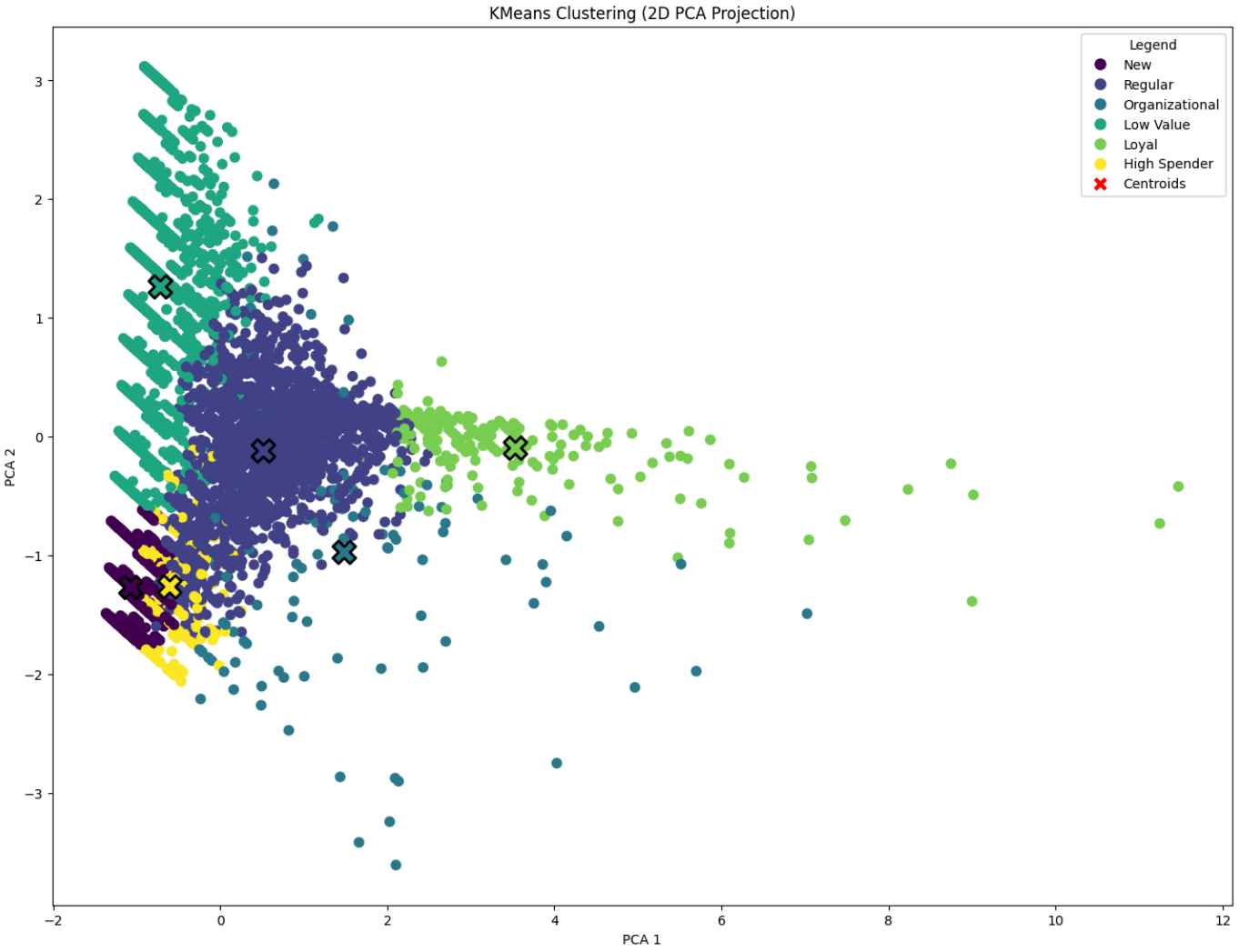
```
labels = Plot_metrics['Cluster']

fig = plt.figure(figsize=(16,12))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    Plot_metrics.to_numpy()[:, 1],
    Plot_metrics.to_numpy()[:, 2],
    Plot_metrics.to_numpy()[:, 3],
    c=labels, cmap='viridis', s=50
)

ax.set_xlabel('Recency')
ax.set_ylabel('Frequency')
ax.set_zlabel('Monetary')
ax.set_title('3D KMeans Clustering')

legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=cluster_labels[i],
           markerfacecolor=scatter.cmap(scatter.norm(i)), markersize=10)
    for i in range(Plot_metrics['Cluster'].nunique())
]

ax.legend(handles=legend_elements, title="Clusters")
plt.show()
```



In [ ]:

```
X = Plot_metrics[['Recency', 'Frequency', 'Monetary', 'First_purchase']]

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

Plot_metrics['PCA1'] = X_pca[:,0]
Plot_metrics['PCA2'] = X_pca[:,1]

cluster_centers = Plot_metrics.groupby('Cluster')[['PCA1','PCA2']].mean().reset_index()

plt.figure(figsize=(16,12))

scatter = plt.scatter(
    Plot_metrics['PCA1'], Plot_metrics['PCA2'], c=Plot_metrics['Cluster'], cmap='viridis', s=
)

plt.scatter(
    cluster_centers['PCA1'], cluster_centers['PCA2'],
    c=cluster_centers['Cluster'], cmap='viridis', marker='X', s=300, label='Centroids', edgec
)
```

```
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('KMeans Clustering (2D PCA Projection)')

legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=cluster_labels[i],
           markerfacecolor=scatter.cmap(scatter.norm(i)), markersize=10)
    for i in range(Plot_metrics['Cluster'].nunique())
]

legend_elements.append(
    Line2D([0], [0], marker='X', color='w', label='Centroids',
           markerfacecolor='red', markersize=12)
)

plt.legend(handles=legend_elements, title="Legend")
plt.show()
```



Clustering alternative

An alternative to clustering, if we want to take a more strict, predetermined, and measureable approach, is to rank our customers based on their RFM values and our own selected criteria.

In my approach, I am going to calculate and assign an aggregate RFM score to each customer, based on how their individual RFM values rank up to the rest.

In [ ]:
```
RFM_scores = RFM.drop(columns='Cluster').copy()

RFM_scores['R_rank'] = RFM_scores['Recency'].rank(ascending=False)
RFM_scores['F_rank'] = RFM_scores['Frequency'].rank(ascending=True)
RFM_scores['M_rank'] = RFM_scores['Monetary'].rank(ascending=True)
```

```
RFM_scores
```

Out[ ]:

	CustomerID	Recency	Frequency	Monetary	First_purchase	Total_Spent	R_rank	F_rank	M_r
0	12347.0	0	7	615.714286	12	4310.00	3500.5	3782.5	37
1	12348.0	2	4	359.310000	11	1437.24	1691.0	3114.0	27
2	12349.0	0	1	1457.550000	0	1457.55	3500.5	777.0	42
3	12350.0	10	1	294.400000	10	294.40	211.0	777.0	21
4	12352.0	1	7	197.962857	9	1385.74	2309.5	3782.5	12
...	...	...	...	...	...	...	...	...	
4313	18280.0	9	1	180.600000	9	180.60	328.5	777.0	10
4314	18281.0	6	1	80.820000	6	80.82	781.5	777.0	1
4315	18282.0	0	2	89.025000	4	178.05	3500.5	1987.0	1
4316	18283.0	0	14	145.684286	11	2039.58	3500.5	4164.5	6
4317	18287.0	1	3	612.426667	6	1837.28	2309.5	2669.0	37

4318 rows × 9 columns

In [ ]:
```
RFM_scores['R_rank_norm'] = (RFM_scores['R_rank'] / RFM_scores['R_rank'].max()) * 100
RFM_scores['F_rank_norm'] = (RFM_scores['F_rank'] / RFM_scores['F_rank'].max()) * 100
RFM_scores['M_rank_norm'] = (RFM_scores['M_rank'] / RFM_scores['M_rank'].max()) * 100

RFM_scores
```

Out[ ]:

	CustomerID	Recency	Frequency	Monetary	First_purchase	Total_Spent	R_rank	F_rank	M_r
0	12347.0	0	7	615.714286	12	4310.00	3500.5	3782.5	37
1	12348.0	2	4	359.310000	11	1437.24	1691.0	3114.0	27
2	12349.0	0	1	1457.550000	0	1457.55	3500.5	777.0	42
3	12350.0	10	1	294.400000	10	294.40	211.0	777.0	21
4	12352.0	1	7	197.962857	9	1385.74	2309.5	3782.5	12
...	...	...	...	...	...	...	...	...	
4313	18280.0	9	1	180.600000	9	180.60	328.5	777.0	10
4314	18281.0	6	1	80.820000	6	80.82	781.5	777.0	1
4315	18282.0	0	2	89.025000	4	178.05	3500.5	1987.0	1
4316	18283.0	0	14	145.684286	11	2039.58	3500.5	4164.5	6
4317	18287.0	1	3	612.426667	6	1837.28	2309.5	2669.0	37

4318 rows × 12 columns

In [ ]:
```
RFM_scores['RFM_Score'] = RFM_scores['R_rank_norm'] + RFM_scores['F_rank_norm'] + RFM_scores[
RFM_scores['RFM_Score'] *= 0.33
RFM_scores = RFM_scores.round(2)
RFM_scores
```

Out[ ]:

	CustomerID	Recency	Frequency	Monetary	First_purchase	Total_Spent	R_rank	F_rank	M_ran
0	12347.0	0	7	615.71	12	4310.00	3500.5	3782.5	3750
1	12348.0	2	4	359.31	11	1437.24	1691.0	3114.0	2746
2	12349.0	0	1	1457.55	0	1457.55	3500.5	777.0	4224
3	12350.0	10	1	294.40	10	294.40	211.0	777.0	2101
4	12352.0	1	7	197.96	9	1385.74	2309.5	3782.5	1256
...	...	...	...	...	...	...	...	...	
4313	18280.0	9	1	180.60	9	180.60	328.5	777.0	1070
4314	18281.0	6	1	80.82	6	80.82	781.5	777.0	162
4315	18282.0	0	2	89.02	4	178.05	3500.5	1987.0	190
4316	18283.0	0	14	145.68	11	2039.58	3500.5	4164.5	675
4317	18287.0	1	3	612.43	6	1837.28	2309.5	2669.0	3740

4318 rows × 13 columns

We now have the Recency, Frequency and Monetary values for each CustomerID, as well as an overall RFM score.

We can now go ahead and assign each Customer to a segment based on their RFM score. For the selection of the segments themselves I am going to be picking four different quantile values from the data.

In [ ]:
```
q95 = RFM_scores['RFM_Score'].quantile(0.95)
q80 = RFM_scores['RFM_Score'].quantile(0.80)
q20 = RFM_scores['RFM_Score'].quantile(0.20)

RFM_scores['RFM_Segment'] = np.where(RFM_scores['RFM_Score'] >= q95, 'Loyal Customer',
                                     np.where(RFM_scores['RFM_Score'] >= q80, 'Regular/High s
                                              np.where(RFM_scores['RFM_Score'] >= q20, 'New/O
RFM_scores = RFM_scores.reset_index()
RFM_scores
```

Out[ ]:

	index	CustomerID	Recency	Frequency	Monetary	First_purchase	Total_Spent	R_rank	F_rank
0	0	12347.0	0	7	615.71	12	4310.00	3500.5	3782.5
1	1	12348.0	2	4	359.31	11	1437.24	1691.0	3114.0
2	2	12349.0	0	1	1457.55	0	1457.55	3500.5	777.0
3	3	12350.0	10	1	294.40	10	294.40	211.0	777.0
4	4	12352.0	1	7	197.96	9	1385.74	2309.5	3782.5
...	...	...	...	...	...	...	...	...	...
4313	4313	18280.0	9	1	180.60	9	180.60	328.5	777.0
4314	4314	18281.0	6	1	80.82	6	80.82	781.5	777.0
4315	4315	18282.0	0	2	89.02	4	178.05	3500.5	1987.0
4316	4316	18283.0	0	14	145.68	11	2039.58	3500.5	4164.5
4317	4317	18287.0	1	3	612.43	6	1837.28	2309.5	2669.0

4318 rows × 15 columns

Now we can calculate RFM statistics, such as the mean and median, monetary percentage, for each different rank.

In [ ]:

```
segment_order = ["Loyal Customer", "Regular/High spender", "New/Occasional Shopper", "Low spe

RFM_scores["RFM_Segment"] = pd.Categorical(RFM_scores["RFM_Segment"], categories=segment_orde

segment_summary = (
    RFM_scores
    .groupby("RFM_Segment", observed=False)[["Recency", "Frequency", "Monetary"]]
    .agg(["mean", "median"])
    .round(2)
)

segment_summary.columns = [
    f"{metric}_{stat}" for metric, stat in segment_summary.columns
]
segment_summary = segment_summary.reset_index()

RFM_scores['Total_Spent'] = RFM_scores['Monetary'] * RFM_scores['Frequency']

Total_revenue = RFM_scores['Total_Spent'].sum()

Revenue_percentage = (RFM_scores.groupby('RFM_Segment', observed=False)['Total_Spent'].sum() /

segment_summary['Revenue_percentage'] = Revenue_percentage
segment_summary['Customer_count'] = RFM_scores['RFM_Segment'].groupby(RFM_scores['RFM_Segment
segment_summary['Customer_percentage'] = (segment_summary['Customer_count'] / segment_summary

segment_summary
```

Out[ ]:

	RFM_Segment	Recency_mean	Recency_median	Frequency_mean	Frequency_median	Monetary_me
0	Loyal Customer	0.00	0.0	13.40	11.0	918
1	Regular/High spender	0.12	0.0	7.27	6.0	495
2	New/Occasional Shopper	1.97	1.0	2.85	2.0	384
3	Low spender	7.12	7.0	1.18	1.0	176

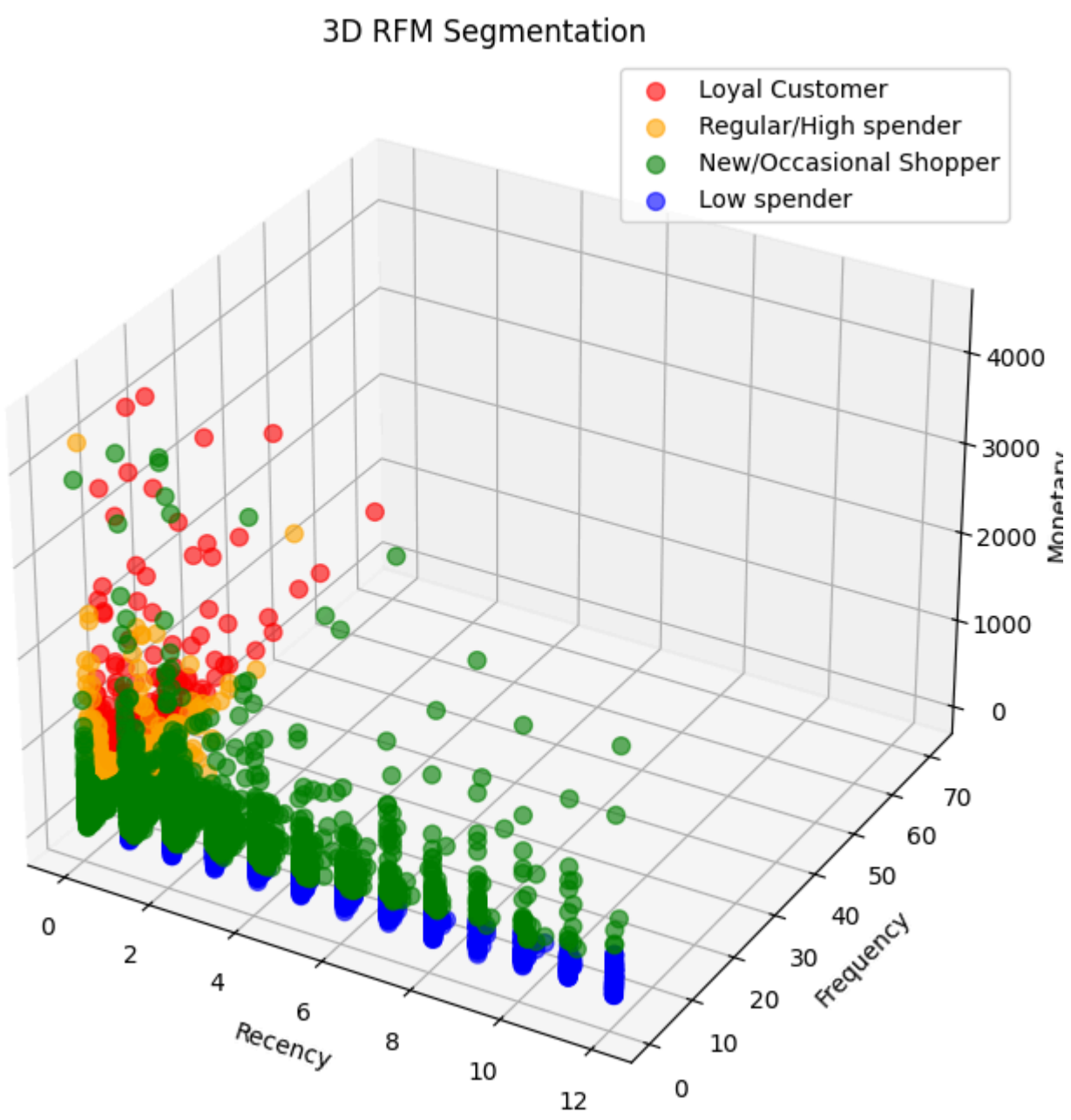
We can note here that the 'Loyal Customers' contributed 37% of the total revenue while representing only 5% of all the customers.

In contrast, the 'Low spender' customers contributed less than 3% of the total revenue while representing 20% of the total customers.

The resulting segments are similar to those we found earlier through clustering.

In [ ]:

```
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# Color mapping for segments
colors = {
    "Loyal Customer": "red",
    "Regular/High spender": "orange",
    "New/Occasional Shopper": "green",
    "Low spender": "blue"
}

for segment, data in RFM_scores.groupby("RFM_Segment", observed=False):
    ax.scatter(
        data['Recency'],
        data['Frequency'],
        data['Monetary'],
        label=segment,
        alpha=0.6,
        s=50,
        c=colors.get(segment, "gray")
    )

ax.set_xlabel("Recency")
ax.set_ylabel("Frequency")
ax.set_zlabel("Monetary")
ax.set_title("3D RFM Segmentation")

ax.legend()
plt.show()
```



Market Basket Analysis

Market Basket Analysis is a data mining technique used to uncover associations and co-occurrence patterns between items in transactional data.

The question it answers is: “What products do customers tend to buy together?”

It’s widely used in retail, e-commerce, and marketing to understand customer purchase behavior.

Its applications include Product placement, Cross-selling / Recommendations and Promotions. Different customer groups may have different frequent itemsets.

I am going to perform market basket analysis using the FP-growth algorithm, for each cluster individually, finding the top 10 best frequent itemsets for each, as well as selectivelly combining the results to find common ground between them.

Prepare the data

In [ ]:
```
df = df.merge(
    RFM[['CustomerID', 'Cluster']],
    on='CustomerID',
```

```
    how='left'
)
df.rename(columns={'Cluster': 'Customer_type'}, inplace=True)
```

In [ ]:
```
filtered_df = df[(df['Revenue'] > 0) & (df['CustomerID'].notna())].copy()
```

Find 10 frequent itemsets for each different customer type

'Loyal' Customers

In [ ]:
```
filtered_loyal = filtered_df[filtered_df['Customer_type'] == 'Loyal']

#Create a 2d matrix with transactions and stockCodes, fill with zeros
basket = (filtered_loyal
          .groupby(['InvoiceNo', 'StockCode'])['Quantity']
          .sum().unstack().fillna(0))

#Convert quantity values to 1s
basket = basket.map(lambda x: 1 if x > 0 else 0)


#Generate itemsets with FP-Growth
frequent_itemsets = fpgrowth(basket, min_support=0.02, use_colnames=True, max_len=3)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=3)

# Sort by highest lift
rules = rules.sort_values("lift", ascending=False)

# Map StockCode → most common Description
stock_desc_map = (filtered_loyal.groupby('StockCode')['Description']
                  .agg(lambda x: x.mode().iloc[0] if not x.mode().empty else None))
```

In [ ]:
```
#Convert stockCodes into Descriptions
def decode_items(itemset):
    return [stock_desc_map.get(x, x) for x in itemset]
```

In [ ]:
```
rules['antecedents'] = rules['antecedents'].apply(lambda x: decode_items(list(x)))
rules['consequents'] = rules['consequents'].apply(lambda x: decode_items(list(x)))
```

In [ ]:
```
rules_loyal = rules[['antecedents','consequents']][(rules['confidence'] >= 0.6) & (rules['lev
rules_loyal.head(10)
```

Out[ ]:

	antecedents	consequents
115	[GREEN REGENCY TEACUP AND SAUCER]	[PINK REGENCY TEACUP AND SAUCER, ROSES REGENCY...
114	[PINK REGENCY TEACUP AND SAUCER, ROSES REGENCY...	[GREEN REGENCY TEACUP AND SAUCER]
116	[PINK REGENCY TEACUP AND SAUCER]	[GREEN REGENCY TEACUP AND SAUCER, ROSES REGENC...
113	[GREEN REGENCY TEACUP AND SAUCER, ROSES REGENC...	[PINK REGENCY TEACUP AND SAUCER]
108	[GREEN REGENCY TEACUP AND SAUCER]	[PINK REGENCY TEACUP AND SAUCER]
109	[PINK REGENCY TEACUP AND SAUCER]	[GREEN REGENCY TEACUP AND SAUCER]
159	[GARDENERS KNEELING PAD KEEP CALM]	[GARDENERS KNEELING PAD CUP OF TEA]
158	[GARDENERS KNEELING PAD CUP OF TEA]	[GARDENERS KNEELING PAD KEEP CALM]
112	[GREEN REGENCY TEACUP AND SAUCER, PINK REGENCY...	[ROSES REGENCY TEACUP AND SAUCER]
74	[GREEN REGENCY TEACUP AND SAUCER]	[ROSES REGENCY TEACUP AND SAUCER]

'Regular' Customers

In [ ]:

```
filtered_regular = filtered_df[filtered_df['Customer_type'] == 'Regular']

#Create a 2d matrix with transactions and stockCodes, fill with zeros
basket = (filtered_regular
          .groupby(['InvoiceNo', 'StockCode'])['Quantity']
          .sum().unstack().fillna(0))

#Convert quantity values to 1s
basket = basket.map(lambda x: 1 if x > 0 else 0)


#Generate itemsets with FP-Growth
frequent_itemsets = fpgrowth(basket, min_support=0.02, use_colnames=True, max_len=3)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=3)

# Sort by highest lift
rules = rules.sort_values("lift", ascending=False)

# Map StockCode → most common Description
stock_desc_map = (filtered_regular.groupby('StockCode')['Description']
                  .agg(lambda x: x.mode().iloc[0] if not x.mode().empty else None))

rules['antecedents'] = rules['antecedents'].apply(lambda x: decode_items(list(x)))
rules['consequents'] = rules['consequents'].apply(lambda x: decode_items(list(x)))

rules_regular = rules[['antecedents','consequents']][(rules['confidence'] >= 0.6) & (rules['l
rules_regular.head(10)
```

Out[ ]:

	antecedents	consequents
72	[GREEN REGENCY TEACUP AND SAUCER]	[PINK REGENCY TEACUP AND SAUCER]
73	[PINK REGENCY TEACUP AND SAUCER]	[GREEN REGENCY TEACUP AND SAUCER]
58	[GREEN REGENCY TEACUP AND SAUCER]	[ROSES REGENCY TEACUP AND SAUCER]
59	[ROSES REGENCY TEACUP AND SAUCER]	[GREEN REGENCY TEACUP AND SAUCER]
70	[SPACEBOY LUNCH BOX]	[DOLLY GIRL LUNCH BOX]
71	[DOLLY GIRL LUNCH BOX]	[SPACEBOY LUNCH BOX]
95	[GARDENERS KNEELING PAD KEEP CALM]	[GARDENERS KNEELING PAD CUP OF TEA]
94	[GARDENERS KNEELING PAD CUP OF TEA]	[GARDENERS KNEELING PAD KEEP CALM]
0	[ALARM CLOCK BAKELIKE GREEN]	[ALARM CLOCK BAKELIKE RED]
1	[ALARM CLOCK BAKELIKE RED]	[ALARM CLOCK BAKELIKE GREEN]

'High Spender' Customers

In [ ]:

```
filtered_high_spender = filtered_df[filtered_df['Customer_type'] == 'High Spender']

#Create a 2d matrix with transactions and stockCodes, fill with zeros
basket = (filtered_high_spender
          .groupby(['InvoiceNo', 'StockCode'])['Quantity']
          .sum().unstack().fillna(0))

#Convert quantity values to 1s
basket = basket.map(lambda x: 1 if x > 0 else 0)


#Generate itemsets with FP-Growth
frequent_itemsets = fpgrowth(basket, min_support=0.02, use_colnames=True, max_len=3)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=3)

# Sort by highest lift
rules = rules.sort_values("lift", ascending=False)

# Map StockCode → most common Description
stock_desc_map = (filtered_high_spender.groupby('StockCode')['Description']
                  .agg(lambda x: x.mode().iloc[0] if not x.mode().empty else None))

rules['antecedents'] = rules['antecedents'].apply(lambda x: decode_items(list(x)))
rules['consequents'] = rules['consequents'].apply(lambda x: decode_items(list(x)))

rules_high_spender = rules[['antecedents','consequents']][(rules['confidence'] >= 0.6) & (rul
rules_high_spender.head(10)
```

Out[ ]:

	antecedents	consequents
2327	[CHILDRENS CUTLERY POLKADOT PINK]	[CHILDRENS CUTLERY POLKADOT BLUE]
2124	[GREEN VINTAGE SPOT BEAKER]	[RED VINTAGE SPOT BEAKER]
2127	[BLUE VINTAGE SPOT BEAKER]	[PINK VINTAGE SPOT BEAKER]
2126	[PINK VINTAGE SPOT BEAKER]	[BLUE VINTAGE SPOT BEAKER]
2125	[RED VINTAGE SPOT BEAKER]	[GREEN VINTAGE SPOT BEAKER]
1891	[MINI JIGSAW DOLLY GIRL]	[MINI JIGSAW SPACEBOY]
1890	[MINI JIGSAW SPACEBOY]	[MINI JIGSAW DOLLY GIRL]
3202	[DECORATIVE WICKER HEART MEDIUM, IVORY WICKER ...	[IVORY WICKER HEART MEDIUM]
3207	[IVORY WICKER HEART MEDIUM]	[DECORATIVE WICKER HEART MEDIUM, IVORY WICKER ...
2326	[CHILDRENS CUTLERY POLKADOT BLUE]	[CHILDRENS CUTLERY POLKADOT PINK]

'New' Customers

In [ ]:

```
filtered_new = filtered_df[filtered_df['Customer_type'] == 'New']

#Create a 2d matrix with transactions and stockCodes, fill with zeros
basket = (filtered_new
          .groupby(['InvoiceNo', 'StockCode'])['Quantity']
          .sum().unstack().fillna(0))

#Convert quantity values to 1s
basket = basket.map(lambda x: 1 if x > 0 else 0)


#Generate itemsets with FP-Growth
frequent_itemsets = fpgrowth(basket, min_support=0.02, use_colnames=True, max_len=3)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=3)

# Sort by highest lift
rules = rules.sort_values("lift", ascending=False)

# Map StockCode → most common Description
stock_desc_map = (filtered_new.groupby('StockCode')['Description']
                  .agg(lambda x: x.mode().iloc[0] if not x.mode().empty else None))

rules['antecedents'] = rules['antecedents'].apply(lambda x: decode_items(list(x)))
rules['consequents'] = rules['consequents'].apply(lambda x: decode_items(list(x)))

rules_new = rules[['antecedents','consequents']][(rules['confidence'] >= 0.6) & (rules['lever
rules_new.head(10)
```

Out[ ]:

	antecedents	consequents
6	[GREEN REGENCY TEACUP AND SAUCER]	[PINK REGENCY TEACUP AND SAUCER]
7	[PINK REGENCY TEACUP AND SAUCER]	[GREEN REGENCY TEACUP AND SAUCER]
24	[SET OF 6 SNACK LOAF BAKING CASES]	[SET OF 12 MINI LOAF BAKING CASES]
25	[SET OF 12 MINI LOAF BAKING CASES]	[SET OF 6 SNACK LOAF BAKING CASES]
48	[SET OF 3 WOODEN TREE DECORATIONS]	[SET OF 3 WOODEN STOCKING DECORATION]
49	[SET OF 3 WOODEN STOCKING DECORATION]	[SET OF 3 WOODEN TREE DECORATIONS]
23	[CHRISTMAS CRAFT WHITE FAIRY]	[CHRISTMAS CRAFT LITTLE FRIENDS]
33	[SET OF 12 MINI LOAF BAKING CASES]	[SET OF 12 FAIRY CAKE BAKING CASES]
19	[HAND WARMER RED LOVE HEART]	[HAND WARMER SCOTTY DOG DESIGN]
14	[HAND WARMER SCOTTY DOG DESIGN]	[HAND WARMER BIRD DESIGN]

'Organizational' Customers

In [ ]:

```
filtered_organizational = filtered_df[filtered_df['Customer_type'] == 'Organizational']

#Create a 2d matrix with transactions and stockCodes, fill with zeros
basket = (filtered_organizational
          .groupby(['InvoiceNo', 'StockCode'])['Quantity']
          .sum().unstack().fillna(0))

#Convert quantity values to 1s
basket = basket.map(lambda x: 1 if x > 0 else 0)


#Generate itemsets with FP-Growth
frequent_itemsets = fpgrowth(basket, min_support=0.02, use_colnames=True, max_len=3)

# Generate association rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=3)

# Sort by highest lift
rules = rules.sort_values("lift", ascending=False)

# Map StockCode → most common Description
stock_desc_map = (filtered_organizational.groupby('StockCode')['Description']
                  .agg(lambda x: x.mode().iloc[0] if not x.mode().empty else None))

rules['antecedents'] = rules['antecedents'].apply(lambda x: decode_items(list(x)))
rules['consequents'] = rules['consequents'].apply(lambda x: decode_items(list(x)))

rules_organizational = rules[['antecedents','consequents']][(rules['confidence'] >= 0.6) & (r
rules_organizational.head(10)
```

Out[ ]:

	antecedents	consequents
100394	[SET/4 RED MINI ROSE CANDLE IN BOWL, PAPER CHA...	[CHRISTMAS GINGHAM TREE]
100395	[CHRISTMAS GINGHAM TREE]	[SET/4 RED MINI ROSE CANDLE IN BOWL, PAPER CHA...
137084	[TRADITIONAL WOODEN CATCH CUP GAME , REINDEER ...	[LAUREL HEART ANTIQUE SILVER]
137079	[LAUREL HEART ANTIQUE SILVER]	[SET OF 6 SOLDIER SKITTLES, AIRLINE LOUNGE,MET...
100866	[CHRISTMAS GINGHAM TREE]	[CHRISTMAS GINGHAM STAR, PAPER CHAIN KIT 50'S ...
100865	[CHRISTMAS GINGHAM STAR]	[CHRISTMAS GINGHAM TREE, PAPER CHAIN KIT 50'S ...
100481	[SET/4 RED MINI ROSE CANDLE IN BOWL]	[SMALL DOLLY MIX DESIGN ORANGE BOWL, CHRISTMAS...
100818	[JAM MAKING SET PRINTED, CHRISTMAS GINGHAM TREE]	[SET/4 RED MINI ROSE CANDLE IN BOWL]
100819	[JAM MAKING SET PRINTED, SET/4 RED MINI ROSE C...	[CHRISTMAS GINGHAM TREE]
100522	[SET/4 RED MINI ROSE CANDLE IN BOWL]	[CHRISTMAS GINGHAM TREE, FEATHER PEN,HOT PINK]

Check for common frequent itemsets between customer types

'Loyal' + 'Regular'

In [ ]:

```
# Create sets of tuples (antecedent, consequent)
set_loyal = set(zip(rules_loyal['antecedents'].apply(frozenset), rules_loyal['consequents'].a
set_regular = set(zip(rules_regular['antecedents'].apply(frozenset), rules_regular['consequen

# Intersection = common rules
common_rules = set_loyal.intersection(set_regular)

# Convert back to DataFrame for readability
common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(ALARM CLOCK BAKELIKE RED)	(ALARM CLOCK BAKELIKE GREEN)
1	(DOLLY GIRL LUNCH BOX)	(SPACEBOY LUNCH BOX)
2	(GARDENERS KNEELING PAD KEEP CALM)	(GARDENERS KNEELING PAD CUP OF TEA)
3	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
4	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)
5	(GREEN REGENCY TEACUP AND SAUCER)	(ROSES REGENCY TEACUP AND SAUCER)
6	(ROSES REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
7	(ALARM CLOCK BAKELIKE GREEN)	(ALARM CLOCK BAKELIKE RED)
8	(JUMBO BAG STRAWBERRY)	(JUMBO BAG RED RETROSPOT)
9	(ALARM CLOCK BAKELIKE PINK)	(ALARM CLOCK BAKELIKE RED)

'New' + 'Regular'

In [ ]:
```
# Create sets of tuples (antecedent, consequent)
set_new= set(zip(rules_new['antecedents'].apply(frozenset), rules_new['consequents'].apply(fr

# Intersection = common rules
common_rules = set_new.intersection(set_regular)

# Convert back to DataFrame for readability
common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
1	(GREEN REGENCY TEACUP AND SAUCER)	(PINK REGENCY TEACUP AND SAUCER)
2	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)

'High Spender' + 'Regular'

In [ ]:
```
# Create sets of tuples (antecedent, consequent)
set_high_spender = set(zip(rules_high_spender['antecedents'].apply(frozenset), rules_high_spe

# Intersection = common rules
common_rules = set_high_spender.intersection(set_regular)

# Convert back to DataFrame for readability
common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(ALARM CLOCK BAKELIKE RED)	(ALARM CLOCK BAKELIKE GREEN)
1	(DOLLY GIRL LUNCH BOX)	(SPACEBOY LUNCH BOX)
2	(LUNCH BAG DOLLY GIRL DESIGN)	(LUNCH BAG SPACEBOY DESIGN)
3	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
4	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)
5	(GREEN REGENCY TEACUP AND SAUCER)	(ROSES REGENCY TEACUP AND SAUCER)
6	(RED HANGING HEART T-LIGHT HOLDER)	(WHITE HANGING HEART T-LIGHT HOLDER)
7	(ROSES REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
8	(ALARM CLOCK BAKELIKE GREEN)	(ALARM CLOCK BAKELIKE RED)
9	(JUMBO BAG STRAWBERRY)	(JUMBO BAG RED RETROSPOT)

'Loyal' + 'High Spender'

In [ ]:
```
common_rules = set_loyal.intersection(set_high_spender)

common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(ALARM CLOCK BAKELIKE RED)	(ALARM CLOCK BAKELIKE GREEN)
1	(DOLLY GIRL LUNCH BOX)	(SPACEBOY LUNCH BOX)
2	(GREEN REGENCY TEACUP AND SAUCER, ROSES REGENC...	(PINK REGENCY TEACUP AND SAUCER)
3	(JUMBO STORAGE BAG SUKI)	(JUMBO BAG RED RETROSPOT)
4	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
5	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)
6	(GREEN REGENCY TEACUP AND SAUCER)	(ROSES REGENCY TEACUP AND SAUCER)
7	(ROSES REGENCY TEACUP AND SAUCER , PINK REGENC...	(GREEN REGENCY TEACUP AND SAUCER)
8	(ROSES REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
9	(GREEN REGENCY TEACUP AND SAUCER, PINK REGENCY...	(ROSES REGENCY TEACUP AND SAUCER)

'Loyal' + 'Regular' + 'New' + 'High Spender'

In [ ]:
```
sets = [set_regular, set_loyal, set_new, set_high_spender]
common_rules = set.intersection(*sets)

common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
1	(GREEN REGENCY TEACUP AND SAUCER)	(PINK REGENCY TEACUP AND SAUCER)
2	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)

'Loyal' + 'Regular' + 'New'

In [ ]:
```
sets = [set_regular, set_loyal, set_new]
common_rules = set.intersection(*sets)

common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
1	(GREEN REGENCY TEACUP AND SAUCER)	(PINK REGENCY TEACUP AND SAUCER)
2	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)

'Loyal' + 'Regular' + 'High Spender'

In [ ]:
```
sets = [set_regular, set_loyal, set_high_spender]
common_rules = set.intersection(*sets)

common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(ALARM CLOCK BAKELIKE RED)	(ALARM CLOCK BAKELIKE GREEN)
1	(DOLLY GIRL LUNCH BOX)	(SPACEBOY LUNCH BOX)
2	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
3	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)
4	(GREEN REGENCY TEACUP AND SAUCER)	(ROSES REGENCY TEACUP AND SAUCER)
5	(ROSES REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
6	(ALARM CLOCK BAKELIKE GREEN)	(ALARM CLOCK BAKELIKE RED)
7	(JUMBO BAG STRAWBERRY)	(JUMBO BAG RED RETROSPOT)
8	(GREEN REGENCY TEACUP AND SAUCER)	(PINK REGENCY TEACUP AND SAUCER)
9	(JUMBO BAG PINK POLKADOT)	(JUMBO BAG RED RETROSPOT)

'Loyal' + 'High Spender' + 'Organizational'

In [ ]:
```
set_organizational = set(zip(rules_organizational['antecedents'].apply(frozenset), rules_orga
sets = [set_loyal, set_organizational, set_high_spender]
common_rules = set.intersection(*sets)

common_rules_df = pd.DataFrame(list(common_rules), columns=['Antecedents', 'Consequents'])
common_rules_df.head(10)
```

Out[ ]:

	Antecedents	Consequents
0	(GREEN REGENCY TEACUP AND SAUCER, ROSES REGENC...	(PINK REGENCY TEACUP AND SAUCER)
1	(JUMBO STORAGE BAG SUKI)	(JUMBO BAG RED RETROSPOT)
2	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
3	(GARDENERS KNEELING PAD CUP OF TEA)	(GARDENERS KNEELING PAD KEEP CALM)
4	(GREEN REGENCY TEACUP AND SAUCER)	(ROSES REGENCY TEACUP AND SAUCER)
5	(ROSES REGENCY TEACUP AND SAUCER , PINK REGENC...	(GREEN REGENCY TEACUP AND SAUCER)
6	(ROSES REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER)
7	(GREEN REGENCY TEACUP AND SAUCER, PINK REGENCY...	(ROSES REGENCY TEACUP AND SAUCER)
8	(PINK REGENCY TEACUP AND SAUCER)	(GREEN REGENCY TEACUP AND SAUCER, ROSES REGENC...
9	(ALARM CLOCK BAKELIKE GREEN)	(ALARM CLOCK BAKELIKE RED)

SQL

Now that I have cleaned and analyzed the data, I'm going to showcase how I would query a database with SQL for simpler analysis that does not require visualisation.

In [ ]:
```
clean_df['Customer_type'] = df['Customer_type']
```

In [ ]:
```
clean_df
```

Out[ ]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
...	...	...	...	...	...	...	...	...
541904	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	2011-12-09 12:50:00	0.85	12680.0	France
541905	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	2011-12-09 12:50:00	2.10	12680.0	France
541906	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	2011-12-09 12:50:00	4.15	12680.0	France
541907	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	2011-12-09 12:50:00	4.15	12680.0	France
541908	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	2011-12-09 12:50:00	4.95	12680.0	France

532492 rows × 11 columns

In [ ]:
```
conn = sqlite3.connect("retail_sales.db")
clean_df.to_sql("sales", conn, if_exists="replace", index=False)
```

Out[ ]: 532492

Top Products by Sales Quantity

In [ ]:
```
query = '''
WITH DescriptionCounts AS (
    SELECT
```

```
        StockCode,
        Description,
        COUNT(*) AS desc_count
    FROM sales
    WHERE Description IS NOT NULL AND UnitPrice > 0
    GROUP BY StockCode, Description
),
MostCommonDescription AS (
    SELECT
        dc.StockCode,
        dc.Description
    FROM DescriptionCounts dc
    JOIN (
        SELECT
            StockCode,
            MAX(desc_count) AS max_count
        FROM DescriptionCounts
        GROUP BY StockCode
    ) m
    ON dc.StockCode = m.StockCode AND dc.desc_count = m.max_count
)
SELECT
    i.StockCode,
    mcd.Description,
    SUM(i.Quantity) AS total_quantity
FROM sales i
LEFT JOIN MostCommonDescription mcd
    ON i.StockCode = mcd.StockCode
WHERE UnitPrice > 0
GROUP BY i.StockCode
ORDER BY total_quantity DESC
LIMIT 10;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	StockCode	Description	total_quantity
0	22197	POPCORN HOLDER	56427
1	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	53751
2	85099B	JUMBO BAG RED RETROSPOT	47256
3	84879	ASSORTED COLOUR BIRD ORNAMENT	36282
4	21212	PACK OF 72 RETROSPOT CAKE CASES	36016
5	85123A	WHITE HANGING HEART T-LIGHT HOLDER	35063
6	23084	RABBIT NIGHT LIGHT	30631
7	22492	MINI PAINT SET VINTAGE	26437
8	22616	PACK OF 12 LONDON TISSUES	26095
9	21977	PACK OF 60 PINK PAISLEY CAKE CASES	24719

These are the products that were sold the most.

Top Products by Revenue

In [ ]:

```
query = '''
WITH RevenuePerProduct AS (
    SELECT
        StockCode,
        SUM(Revenue) AS TotalRevenue
```

```
    FROM sales
    WHERE StockCode IS NOT NULL
    GROUP BY StockCode
),
Descriptions AS (
    SELECT
        StockCode,
        Description,
        COUNT(*) AS freq
    FROM sales
    WHERE Description IS NOT NULL
    GROUP BY StockCode, Description
),
TopDescriptions AS (
    SELECT StockCode, Description
    FROM (
        SELECT
            StockCode,
            Description,
            RANK() OVER (PARTITION BY StockCode ORDER BY freq DESC) AS rnk
        FROM Descriptions
    )
    WHERE rnk = 1
)
SELECT
    r.StockCode,
    d.Description,
    r.TotalRevenue
FROM RevenuePerProduct r
LEFT JOIN TopDescriptions d ON r.StockCode = d.StockCode
ORDER BY r.TotalRevenue DESC
LIMIT 10;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	StockCode	Description	TotalRevenue
0	22423	REGENCY CAKESTAND 3 TIER	164459.49
1	47566	PARTY BUNTING	98243.88
2	85123A	WHITE HANGING HEART T-LIGHT HOLDER	97838.45
3	85099B	JUMBO BAG RED RETROSPOT	92175.79
4	23084	RABBIT NIGHT LIGHT	66661.63
5	22086	PAPER CHAIN KIT 50'S CHRISTMAS	63715.24
6	84879	ASSORTED COLOUR BIRD ORNAMENT	58792.42
7	79321	CHILLI LIGHTS	53746.66
8	22502	PICNIC BASKET WICKER SMALL	51023.52
9	22197	POPCORN HOLDER	50967.92

These are the items that generated the highest revenue.

Top Products by Unique Transactions

In [ ]:

```
query = '''
WITH DescriptionCounts AS (
    SELECT
        StockCode,
        Description,
```

```
        COUNT(*) AS desc_count
    FROM sales
    WHERE Description IS NOT NULL AND Quantity > 0
    GROUP BY StockCode, Description
),
MostCommonDescription AS (
    SELECT
        dc.StockCode,
        dc.Description
    FROM DescriptionCounts dc
    JOIN (
        SELECT
            StockCode,
            MAX(desc_count) AS max_count
        FROM DescriptionCounts
        GROUP BY StockCode
    ) m
    ON dc.StockCode = m.StockCode AND dc.desc_count = m.max_count
)
SELECT
    i.StockCode,
    mcd.Description,
    COUNT(DISTINCT i.InvoiceNo) AS unique_invoices
FROM sales i
LEFT JOIN MostCommonDescription mcd
    ON i.StockCode = mcd.StockCode
WHERE i.UnitPrice > 0
GROUP BY i.StockCode, mcd.Description
ORDER BY unique_invoices DESC
LIMIT 10;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	StockCode	Description	unique_invoices
0	85123A	WHITE HANGING HEART T-LIGHT HOLDER	2240
1	22423	REGENCY CAKESTAND 3 TIER	2168
2	85099B	JUMBO BAG RED RETROSPOT	2132
3	47566	PARTY BUNTING	1705
4	20725	LUNCH BAG RED RETROSPOT	1608
5	84879	ASSORTED COLOUR BIRD ORNAMENT	1467
6	22720	SET OF 3 CAKE TINS PANTRY DESIGN	1458
7	22197	POPCORN HOLDER	1442
8	21212	PACK OF 72 RETROSPOT CAKE CASES	1334
9	22383	LUNCH BAG SUKI DESIGN	1305

These are the items that were included in the most different transactions.

Top Returned Products by Quantity

In [ ]:

```
query = '''
WITH ReturnedItems AS (
    SELECT
        StockCode,
        SUM(ABS(Quantity)) AS TotalReturnedQty
    FROM sales
    WHERE Quantity < 0 AND StockCode IS NOT NULL AND UnitPrice > 0
```

```
    GROUP BY StockCode
),
Descriptions AS (
    SELECT
        StockCode,
        Description,
        COUNT(*) AS freq
    FROM sales
    WHERE Description IS NOT NULL
    GROUP BY StockCode, Description
),
TopDescriptions AS (
    SELECT StockCode, Description
    FROM (
        SELECT
            StockCode,
            Description,
            RANK() OVER (PARTITION BY StockCode ORDER BY freq DESC) AS rnk
        FROM Descriptions
    )
    WHERE rnk = 1
)
SELECT
    r.StockCode,
    d.Description,
    r.TotalReturnedQty
FROM ReturnedItems r
LEFT JOIN TopDescriptions d ON r.StockCode = d.StockCode
ORDER BY r.TotalReturnedQty DESC
LIMIT 10;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	StockCode	Description	TotalReturnedQty
0	84347	ROTATING SILVER ANGELS T-LIGHT HLDR	9376
1	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	3150
2	85123A	WHITE HANGING HEART T-LIGHT HOLDER	2578
3	21175	GIN + TONIC DIET METAL SIGN	2030
4	22920	HERB MARKER BASIL	1527
5	22273	FELTCRAFT DOLL MOLLY	1447
6	47566B	TEA TIME PARTY BUNTING	1424
7	15034	PAPER POCKET TRAVELING FAN	1385
8	20971	PINK BLUE FELT CRAFT TRINKET BOX	1321
9	71477	COLOUR GLASS. STAR T-LIGHT HOLDER	1228

These were the most returned products.

Top Returned Products by Revenue

In [ ]:

```
query = '''
WITH ReturnRevenue AS (
    SELECT
        StockCode,
        SUM(Quantity * UnitPrice) AS ReturnLoss
    FROM sales
    WHERE Quantity < 0 AND StockCode IS NOT NULL
```

```
    GROUP BY StockCode
),
Descriptions AS (
    SELECT
        StockCode,
        Description,
        COUNT(*) AS freq
    FROM sales
    WHERE Description IS NOT NULL
    GROUP BY StockCode, Description
),
TopDescriptions AS (
    SELECT StockCode, Description
    FROM (
        SELECT
            StockCode,
            Description,
            RANK() OVER (PARTITION BY StockCode ORDER BY freq DESC) AS rnk
        FROM Descriptions
    )
    WHERE rnk = 1
)
SELECT
    r.StockCode,
    d.Description,
    ABS(r.ReturnLoss) AS ReturnLossAmount
FROM ReturnRevenue r
LEFT JOIN TopDescriptions d ON r.StockCode = d.StockCode
ORDER BY ReturnLossAmount DESC
LIMIT 10;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	StockCode	Description	ReturnLossAmount
0	22423	REGENCY CAKESTAND 3 TIER	9697.05
1	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6624.30
2	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	6591.42
3	23113	PANTRY CHOPPING BOARD	4803.06
4	48185	DOORMAT FAIRY CAKE	4554.90
5	21175	GIN + TONIC DIET METAL SIGN	3775.33
6	47566B	TEA TIME PARTY BUNTING	3692.95
7	22273	FELTCRAFT DOLL MOLLY	3512.65
8	71477	COLOUR GLASS. STAR T-LIGHT HOLDER	3443.62
9	22191	IVORY DINER WALL CLOCK	2653.70

These products accumulated the largest losses due to returns.

Top Returned Products by unique Transactions

In [ ]:

```
query = '''
WITH DescriptionCounts AS (
    SELECT
        StockCode,
        Description,
        COUNT(*) AS desc_count
    FROM sales
```

```
    WHERE Description IS NOT NULL
    GROUP BY StockCode, Description
),
MostCommonDescription AS (
    SELECT
        dc.StockCode,
        dc.Description
    FROM DescriptionCounts dc
    JOIN (
        SELECT
            StockCode,
            MAX(desc_count) AS max_count
        FROM DescriptionCounts
        GROUP BY StockCode
    ) m
    ON dc.StockCode = m.StockCode AND dc.desc_count = m.max_count
),
ReturnedInvoices AS (
    SELECT DISTINCT
        InvoiceNo,
        StockCode
    FROM sales
    WHERE Quantity < 0  -- returned items
)
SELECT
    r.StockCode,
    mcd.Description,
    COUNT(DISTINCT r.InvoiceNo) AS return_invoices_count
FROM ReturnedInvoices r
LEFT JOIN MostCommonDescription mcd
    ON r.StockCode = mcd.StockCode
GROUP BY r.StockCode, mcd.Description
ORDER BY return_invoices_count DESC
LIMIT 10;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	StockCode	Description	return_invoices_count
0	22423	REGENCY CAKESTAND 3 TIER	183
1	22960	JAM MAKING SET WITH JARS	87
2	22720	SET OF 3 CAKE TINS PANTRY DESIGN	75
3	21232	STRAWBERRY CERAMIC TRINKET BOX	57
4	22699	ROSES REGENCY TEACUP AND SAUCER	54
5	22197	POPCORN HOLDER	50
6	22666	RECIPE BOX PANTRY YELLOW DESIGN	47
7	20725	LUNCH BAG RED RETROSPOT	43
8	21843	RED RETROSPOT CAKE STAND	43
9	85099B	JUMBO BAG RED RETROSPOT	43

These were the most commonly returned products in different transactions.

Top faulty/damaged/thrown away items by quantity

In [ ]:

```
query = '''
WITH DescriptionCounts AS (
    SELECT
```

```
        StockCode,
        Description,
        COUNT(*) AS desc_count
    FROM sales
    WHERE Description IS NOT NULL
    GROUP BY StockCode, Description
),
MostCommonDescription AS (
    SELECT
        dc.StockCode,
        dc.Description
    FROM DescriptionCounts dc
    JOIN (
        SELECT
            StockCode,
            MAX(desc_count) AS max_count
        FROM DescriptionCounts
        GROUP BY StockCode
    ) m
    ON dc.StockCode = m.StockCode AND dc.desc_count = m.max_count
)
SELECT
    i.StockCode,
    mcd.Description,
    COUNT(*) AS count
FROM sales i
LEFT JOIN MostCommonDescription mcd
    ON i.StockCode = mcd.StockCode
WHERE i.Quantity < 0
  AND i.IsCancelled = 0       -- not cancelled
  AND (i.UnitPrice = 0 OR i.CustomerID IS NULL) -- suspicious condition
GROUP BY i.StockCode, mcd.Description
ORDER BY count DESC
LIMIT 10;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	StockCode	Description	count
0	21830	ASSORTED CREEPY CRAWLIES	5
1	85175	CACTI T-LIGHT CANDLES	5
2	22719	GUMBALL MONOCHROME COAT RACK	4
3	72802C	VANILLA SCENT CANDLE JEWELLED BOX	4
4	82494L	WOODEN FRAME ANTIQUE WHITE	4
5	85172	HYACINTH BULB T-LIGHT CANDLES	4
6	20713	JUMBO BAG OWLS	3
7	20774	GREEN FERN NOTEBOOK	3
8	20892	SET/3 TALL GLASS CANDLE HOLDER PINK	3
9	20966	SANDWICH BATH SPONGE	3

These products were the most common faulty or damaged.

Cancellation Rate

In [ ]:

```
query = '''
SELECT
  ROUND(100.0 * SUM(CASE WHEN IsCancelled THEN 1 ELSE 0 END) / COUNT(*), 2) AS cancellation_ra
```

```
FROM sales;
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	cancellation_rate_pct
0	1.63

The total cancellation rate was 1.62%

Return Rate by Country

In [ ]:

```
query = '''
SELECT Country,
       COUNT(*) AS total_orders,
       SUM(CASE WHEN IsCancelled THEN 1 ELSE 0 END) AS cancelled_orders,
       ROUND(100.0 * SUM(CASE WHEN IsCancelled THEN 1 ELSE 0 END) / COUNT(*), 2) AS return_rat
FROM sales
GROUP BY Country
ORDER BY return_rate_pct DESC
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	Country	total_orders	cancelled_orders	return_rate_pct
0	USA	291	112	38.49
1	Czech Republic	28	4	14.29
2	Malta	123	14	11.38
3	Saudi Arabia	10	1	10.00
4	Japan	355	34	9.58
5	Australia	1253	73	5.83
6	Italy	783	42	5.36
7	Bahrain	19	1	5.26
8	Germany	9080	437	4.81
9	EIRE	8059	291	3.61
10	Poland	336	11	3.27
11	Denmark	375	8	2.13
12	Sweden	436	9	2.06
13	Spain	2462	45	1.83
14	Belgium	1971	36	1.83
15	European Community	58	1	1.72
16	Switzerland	1960	33	1.68
17	France	8218	133	1.62
18	United Kingdom	487487	7327	1.50
19	Channel Islands	752	9	1.20
20	Cyprus	608	7	1.15
21	Norway	1059	11	1.04
22	Finland	653	6	0.92
23	Austria	387	3	0.78
24	Portugal	1466	11	0.75
25	Israel	294	2	0.68
26	Hong Kong	276	1	0.36
27	Netherlands	2326	4	0.17
28	Unspecified	442	0	0.00
29	United Arab Emirates	67	0	0.00
30	Singapore	215	0	0.00
31	RSA	57	0	0.00
32	Lithuania	35	0	0.00
33	Lebanon	45	0	0.00
34	Iceland	182	0	0.00
35	Greece	142	0	0.00

	Country	total_orders	cancelled_orders	return_rate_pct
36	Canada	150	0	0.00
37	Brazil	32	0	0.00

This is the full list of return rates per country.

Top Countries by Revenue

In [ ]:
```
query = '''
SELECT Country, ROUND(SUM(Quantity * UnitPrice), 2) AS Revenue
FROM sales
GROUP BY Country
ORDER BY Revenue DESC
'''
pd.read_sql_query(query, conn)
```

Out[ ]:

	Country	Revenue
0	United Kingdom	8280991.91
1	Netherlands	283479.54
2	EIRE	259380.02
3	Germany	200619.66
4	France	182076.60
5	Australia	136922.50
6	Switzerland	52483.05
7	Spain	51746.65
8	Belgium	36662.96
9	Japan	35419.79
10	Sweden	35166.41
11	Norway	32292.96
12	Portugal	26807.47
13	Channel Islands	19926.39
14	Finland	18303.54
15	Denmark	18042.14
16	Italy	15276.34
17	Cyprus	12843.76
18	Hong Kong	9733.24
19	Singapore	9120.39
20	Austria	8698.32
21	Israel	7901.97
22	Poland	6853.14
23	Unspecified	4740.94
24	Greece	4425.52
25	Iceland	4310.00
26	Canada	3115.44
27	Malta	1980.47
28	United Arab Emirates	1864.78
29	USA	1730.92
30	Lebanon	1693.88
31	Lithuania	1661.06
32	European Community	1150.75
33	Brazil	1143.60
34	RSA	1002.31
35	Czech Republic	671.72

	Country	Revenue
36	Bahrain	548.40
37	Saudi Arabia	131.17

This is the full list of revenue per country.